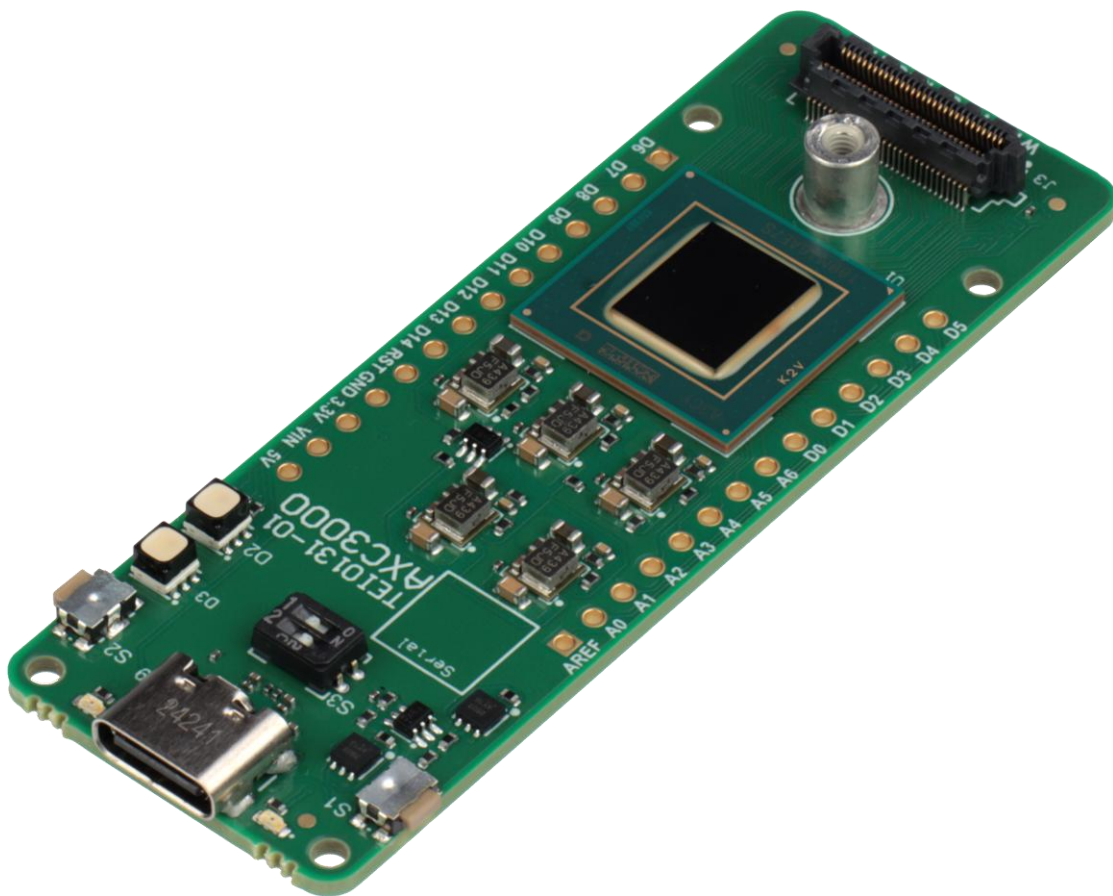




AXC3000 Simple Nios® V Lab



Software and hardware requirements to complete all exercises.

Software Requirements: Quartus® Prime Pro version 25.1.1

Hardware Requirements: AXC3000 Agilex™ 3C Development Kit

Document Control

Document Version:	Version 1.0
Document Date:	10/16/2025
Document Author(s):	Steven Kravatsky, Erika Peter, Naji Naufel, , ESC Team
Document Classification:	Released
Document Distribution:	This document is still under development. All specifications, procedures, and processes described in this document are subject to change without prior notice
Prior Version History:	

Please read the legal disclaimer at the end of this document.

Contents

1	Introduction	4
1.1	Objective	4
2	Getting Started	5
3	Design Flow	6
4	Hardware Project with AXC3000 board	7
4.1	New Quartus Prime project	7
4.2	Design Entry	10
4.3	Design Flow	12
4.4	Complete the Quartus Design	30
5	Software Project with AXC3000 board	35
5.1	Create a New Software Project	35
5.2	Generate the project BSP	35
5.3	Building software project using Ashling RiscFree IDE for Intel FPGA	39
5.4	Compile the Design in Quartus	44
6	Validate the design Locally	46
6.1	File Download	46
6.2	Program the FPGA	47
6.3	Open a Terminal Emulator	50
7	Validate the Design Remotely	52
7.1	Connect to the Board Farm	52
7.2	Program the FPGA	54
7.3	Open a Terminal Emulator	56
8	Legal Disclaimer	58

1 Introduction

This tutorial provides comprehensive information to help you understand how to create a simple Nios® V Design and run it on your board. This lab will not make you an expert, but at the end, you will understand basic concepts about the embedded hardware and software flow targeting the soft core Nios V and downloading the system into the FPGA on your development board.

1.1 Objective

1. Create a Hardware Design
 - a. Simple design based on a Nios V CPU in Platform Designer
 - b. Top-level HDL, and timing constraints
 - c. Assign pins and Compile project
2. Create a Software Design
 - a. Create a Board Support Package (BSP)
 - b. Create a simple C code program
 - c. Compile and Generate the ELF file
3. Download and Run Application on a Development Board

Lab Notes:

Many of the names that the lab asks you to use for files, components, and other objects in this exercise must be spelled exactly as directed. This nomenclature is necessary because the pre-written software application includes variables that use the names of the hardware peripherals. Naming the components differently can cause errors.

This lab can be compiled on a local machine or on a remote machine hosted by CloudLabs. Both options allow for the lab to be tested on a local AXC3000 board.

In addition, when using CloudLabs, testing can also be performed using a remote board in the Arrow board farm. Both options are documented.

2 Getting Started

The first objective is to ensure that you have all the necessary software installed so that the lab can be completed successfully. Below is a list of items required to complete this lab. A number of options are provided.

Option 1: Completing the lab using CloudLabs tools and the board farm.

- Personal computer or laptop running 64-bit Linux / Windows 10 or later with at least an Intel i3 core (or equivalent), 8GB RAM.

Option 2: Completing the lab using CloudLabs tools and local hardware.

- AXC3000 Board
- USB C cable
- Quartus Prime 25.1 .1Pro Standalone Programmer.
- Personal computer or laptop running 64-bit Linux / Windows 10 or later with at least an Intel i3 core (or equivalent), 8GB RAM.
- Terminal emulator (Putty or Tera term)

Option 3: Completing the lab using a laptop and local hardware.

- AXC3000 Board
- USB C cable
- Quartus Prime 25.1.1 Pro
- Ashling RiscFree IDE
- Personal computer or laptop running 64-bit Linux / Windows 10 or later with at least an Intel i3 core (or equivalent), 8GB RAM.
- Terminal emulator (Putty or Tera term)

A desire to learn!

3 Design Flow

Developing software for an embedded system on a programmable chip requires an understanding of the design flow between the Platform Designer system integration tool (hardware) and the Nios V Embedded Development Suite (EDS). Typically, designs begin with requirements that become inputs to system definitions. System definition is the first step in the design flow process. For this workshop, the design will be built and then the FPGA image will be downloaded onto the board. The objective of this module is to review the development tools that will be used.

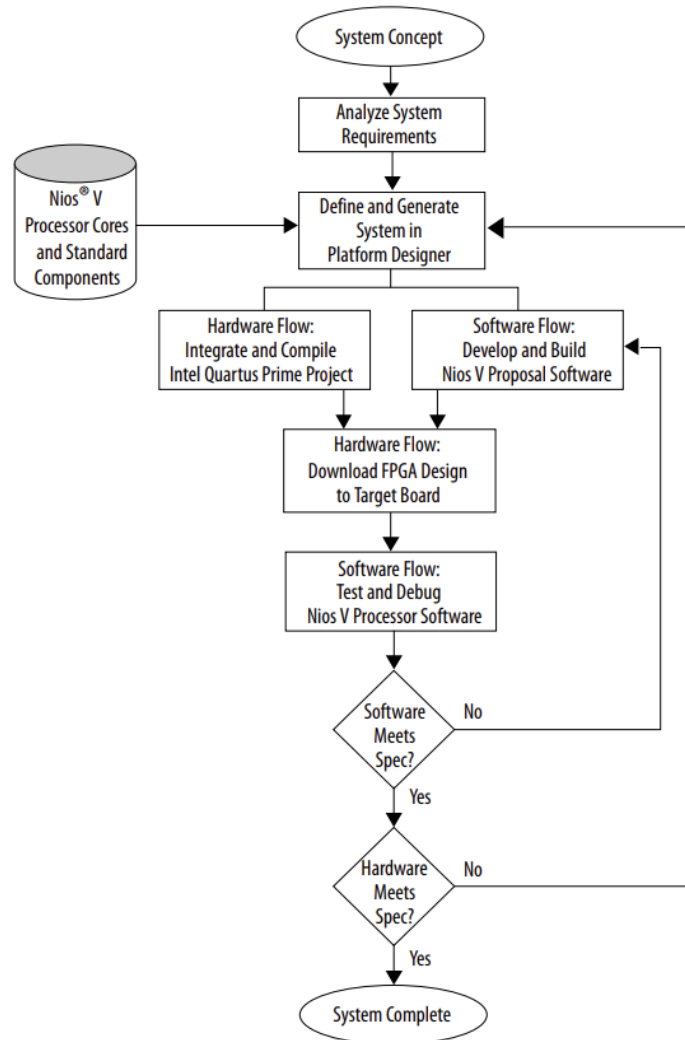


Figure 3-1 : Design Flow

The above diagram shows the typical design flow for the system design. The system definition is done with Platform Designer. The Nios V IDE uses the system description to create a new project for the software application. The output of the FPGA design is an FPGA image that is used to configure the FPGA. The output of the software flow is an executable which runs on the Nios V (soft core) processor.

4 Hardware Project with AXC3000 board

This chapter gives instructions for creating the hardware project in Quartus® Prime Pro 25.1.

4.1 New Quartus Prime project

The hardware implementation starts with a Quartus® Prime project. A Quartus project defines the location for the HDL and IP files used, target family, target device, and initial settings. After the initial creation of the project, changes can be made. Quartus uses the project location for the generated files.

4.1.1 New project creation

There are many screens that will need to be filled in when creating a new project. The steps below will take you through all of them.

4.1.1.1 Open Quartus Pro 25.1.1

If not already open, from the Start menu or the Desktop, open the Quartus Prime Pro 25.1.1 IDE.

4.1.1.2 Create a new project

Using the New Project Wizard: **File** → **New Project Wizard**. This screen shows an introduction window. Click **Next**.

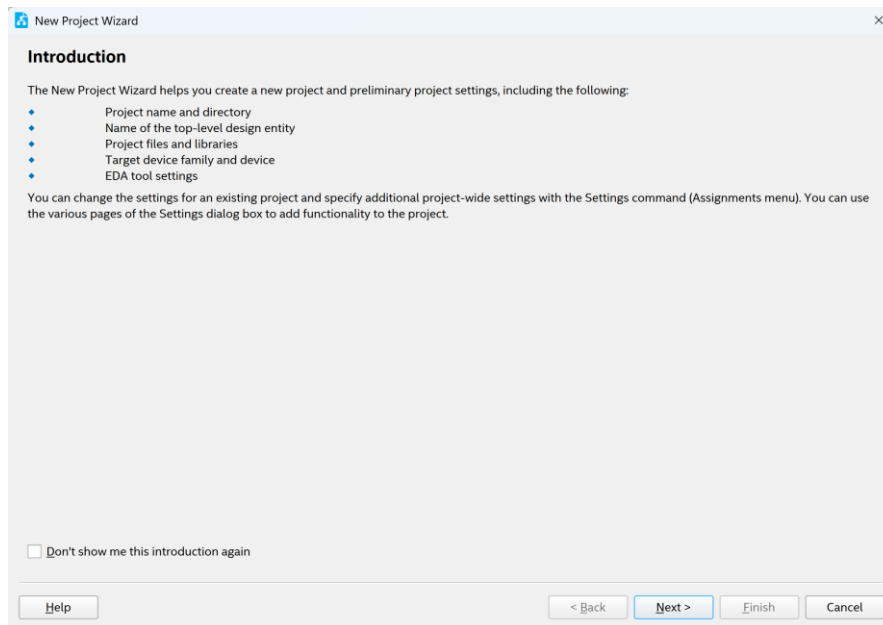


Figure 4-1 : New Project Wizard Introduction

4.1.1.3 Configure the New Project Wizard

Specify project directory, name, and top-level entity information:

Note: No spaces are allowed in path or filenames.

- Ensure that the **Empty Project** radio button is selected on the left side of the window.
- Enter a directory in which you will store your Quartus project files for this design C:/altera_workshops/axc3000/NIOSV_lab
- Specify the name of the project: **NIOSV_lab**
- Specify the name of the top-level entity: **NIOSV_lab**
- Click **Next**.

If you are asked to allow creation of a non-existent folder, click **Yes**.

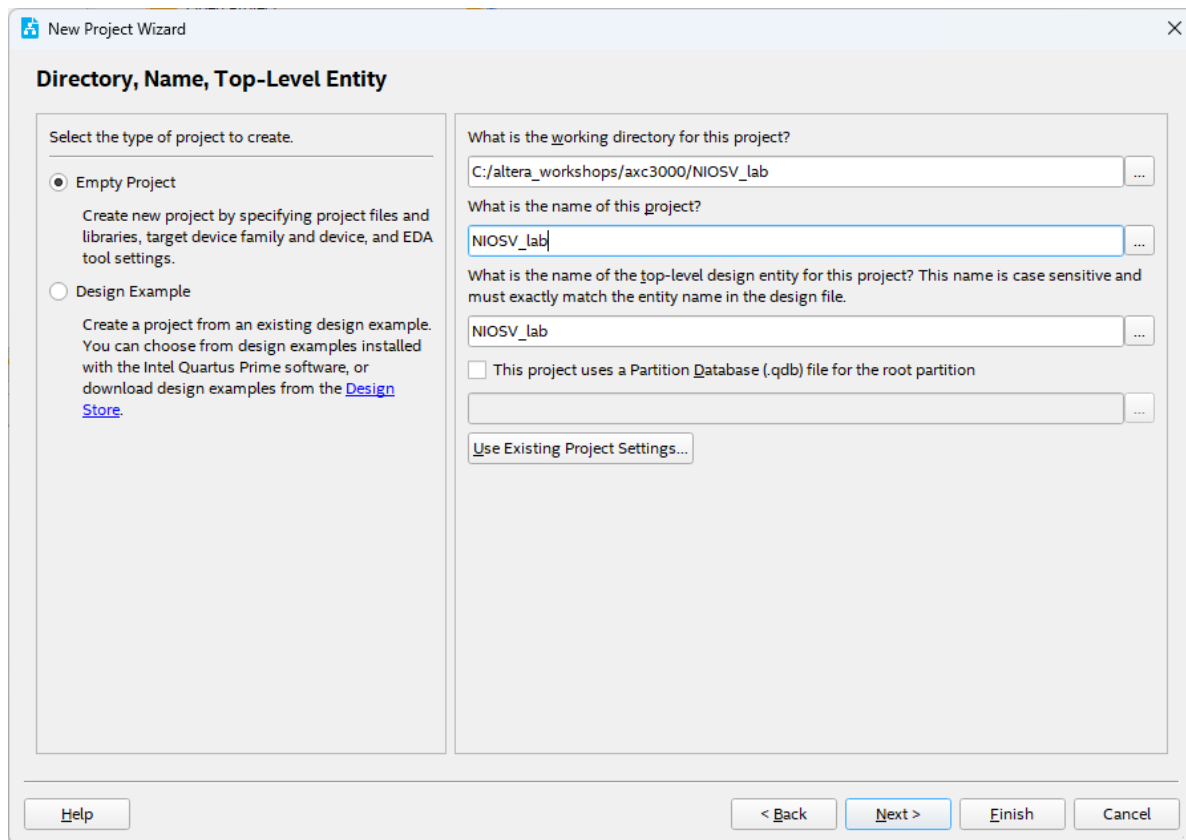


Figure 4-2 : New Project Wizard Folder/Project Names

4.1.1.4 Specify Family and Device Settings

In the Family field, select **Agilex 3 (C-Series)** and enter the part number (**A3CY100BM16AE7S**) in the Name Filter text box and select it. Click **Next**.

Family, Device & Board Settings

Device | Board

Select the family and device you want to target for compilation. You can install additional device support with the Install Devices command c menu.

Device family

Family: Agilex 3 (C-Series) | Device: All

Target device

☒ Specific device selected in 'Available devices' list
☐ Other: n/a

Show in 'Available devices' list

Package: Any
Pin count: Any
Core speed grade: Any
☒ Show advanced devices

Filter: Name | A3CY100BM16AE7S

	Name	Core Voltage	ALMs	Total I/Os	GPIOs	GTS XCVR Channels	Memory Bits	M20K	I
11	A3CY100BM16AE7S	0.75V	34000	254	192	4	5365760	262	13

Figure 4-3 : New Project Wizard Device Selection

4.1.1.5 Add Files

On the Add Files page, click **Next**. No files will be included.

4.1.1.6 EDA Tools Settings

On the EDA Tools Settings page, select **Next**.

4.1.1.7 Summary Screen

The Summary screen will show the file location and the device selection. These settings can be changed if needed, later.

Click **Finish**.

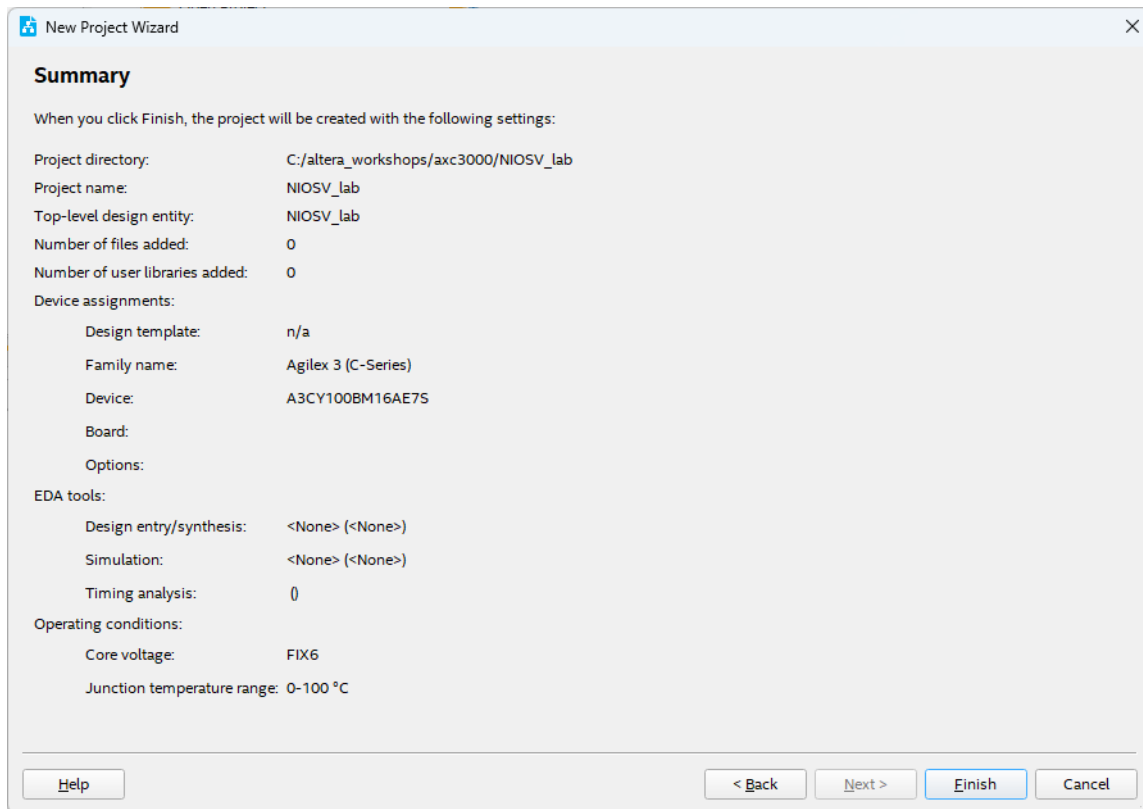


Figure 4-4 : Project Summary

4.2 Design Entry

In this module you will use Platform Designer to create hardware for the processor system. You will add components and make interface connections.

4.2.1 Project Top Level

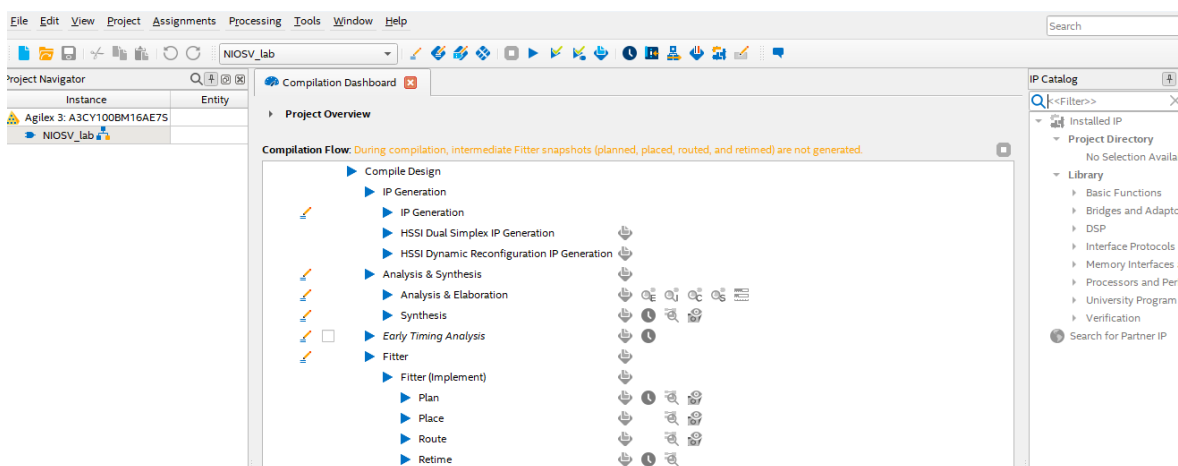


Figure 4-5 : Quartus Project View

The top-level file is seen in the **Project Navigator** pane (top left). The middle section (**Compilation Flow**) shows the FPGA steps which will be taken by the compiler. These steps include analysis and synthesis, fitter and etc.

The **AXC3000** board has an LED indicator on the **CONFIG_DONE** pin. **CONFIG_DONE**, being a Dual-Purpose pin, has to be enabled to drive the LED once the FPGA enters user mode. To do so, select the menu **Assignments -> Device** then click the **Device and Pin Options...** button. When a window opens:

- Select **Configuration** in the Category pane and Click the **Configuration Pin Options** bar.
- Check the box to the left of **USE CONF_DONE** output and select **SDM_I016**
- Check the box to the left of **USE INIT_DONE** output and select **SDM_I00**
- Click **OK** in multiple windows until done with this task.

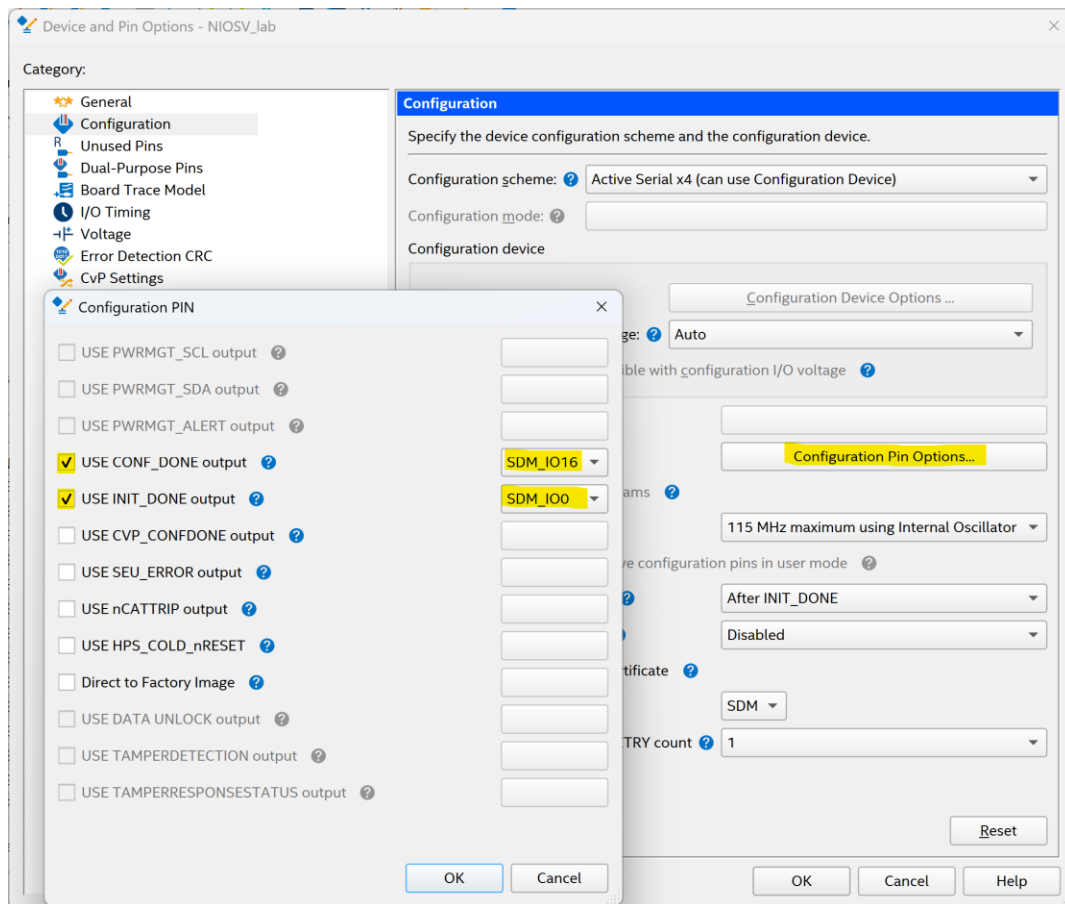


Figure 4-6 : Configuration Pin Assignment

4.3 Design Flow

Our design flow starts with Platform Designer where we add the Nios V CPU and its peripherals used in this workshop.

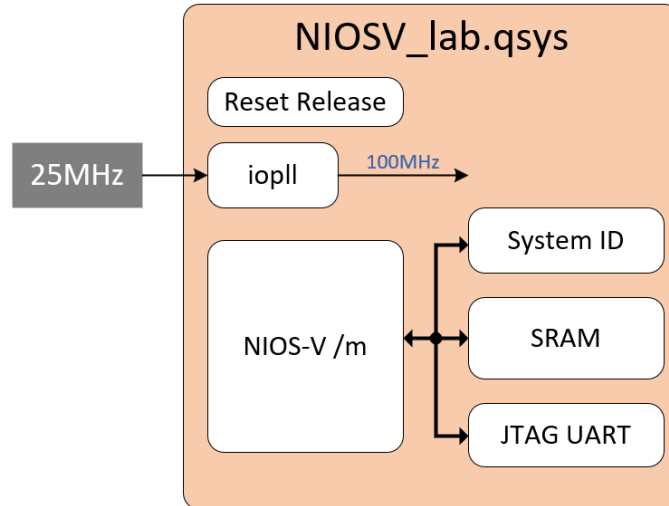


Figure 4-7 : Design Block Diagram

4.3.1 Add components to Platform Designer

Platform Designer is a high-level system integration tool that allows you to quickly build a system using Intel IP block as well as custom components. The tool automatically creates interconnect logic between the components for easy design use.

Platform Designer has various sections and automatically generates the high-performance interconnect between them. It allows you to connect components on an interface level, rather than the signal-by-signal level. The tool understands the different types of interfaces and will only allow connections between interfaces of same type (i.e. a data master connects to a data slave, clock source to clock sink and etc ...).

4.3.1.1 Open Platform Designer

from the **Tools** → **Platform Designer** or clicking on  button on the toolbar. The **Open System** window pops up.

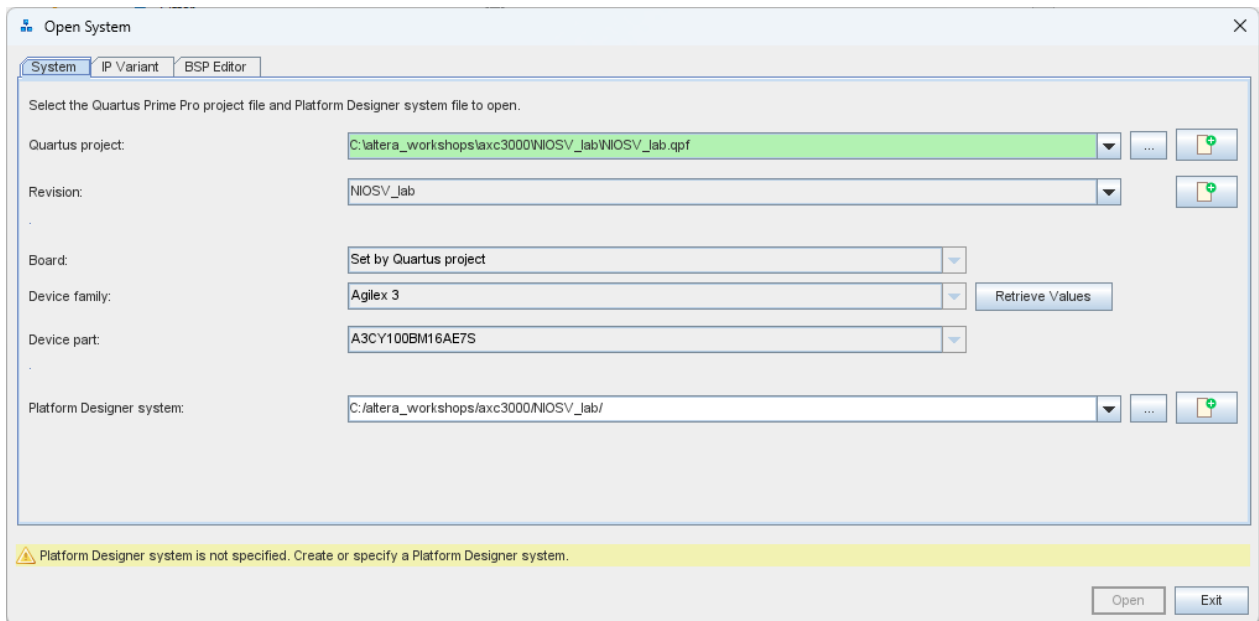


Figure 4-8 : Open System

In the Open System window, click the icon on the Platform Designer system line to create a new system. If the system already exists, click the symbol instead, to open it. In the pop-up window, give it the name, then click the **Create** button.

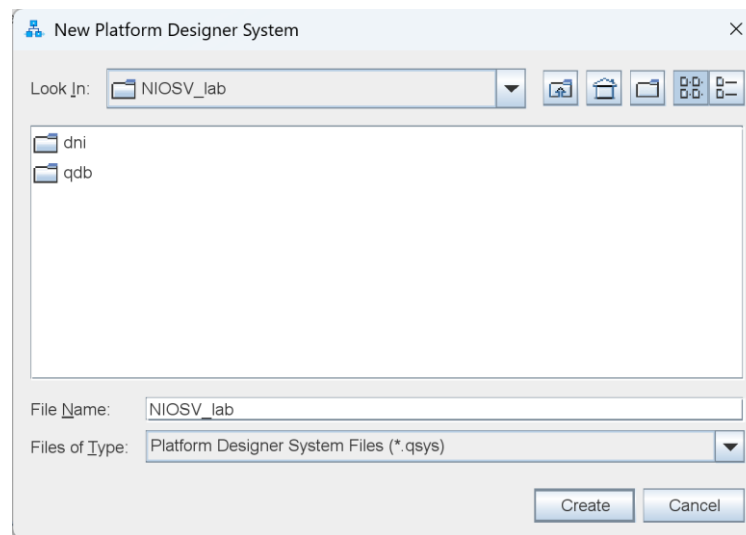


Figure 4-9 : New Platform Designer System window

In the **Open System** window, click the **Create** button.

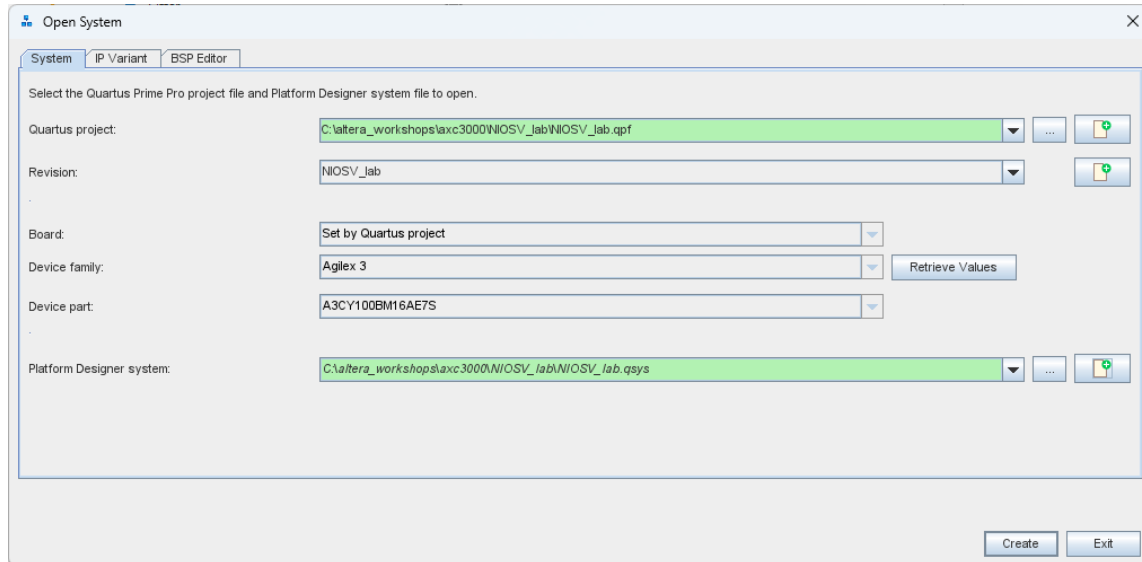


Figure 4-10 : Completed Open System window

When the **Create New System** window shows that it completed successfully, click on the **Close** button.

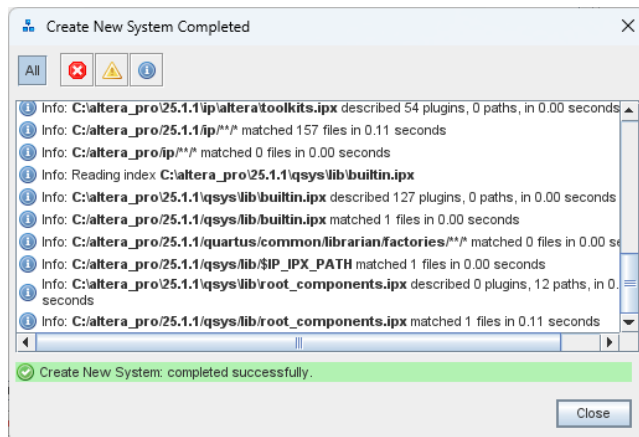


Figure 4-11 : Create New System Completed

In the new window, you should see a single clock source component, named **clock_in** in the **System View** tab. This tab shows all the components currently in your system. There will be 2 components that are automatically part of the Platform Designer, the Clock Bridge and the Reset Bridge.

All FPGA designs need at least one Clock Bridge. Synchronous designs are more reliable, have better performance and are more verifiable than a combinational design. Combinational design is not a good design approach.

The Reset Bridge allows the reset signal to be connected to different components and sub-systems in the Platform Designer. The reset bridge defines how the reset is synchronized (i.e. synchronous/asynchronous) and if the reset is active-low/high.

4.3.1.2 Configure the Clock Bridge

The Clock Bridge has a default clock speed assigned to its input. The frequency needs to be changed to match the clock source on the **AXC3000** board. In this case, 25MHz.

Double-click the **clock_in** IP. In the Parameters window that shows up, edit the **Explicit Clock Rate** to **25000000**.

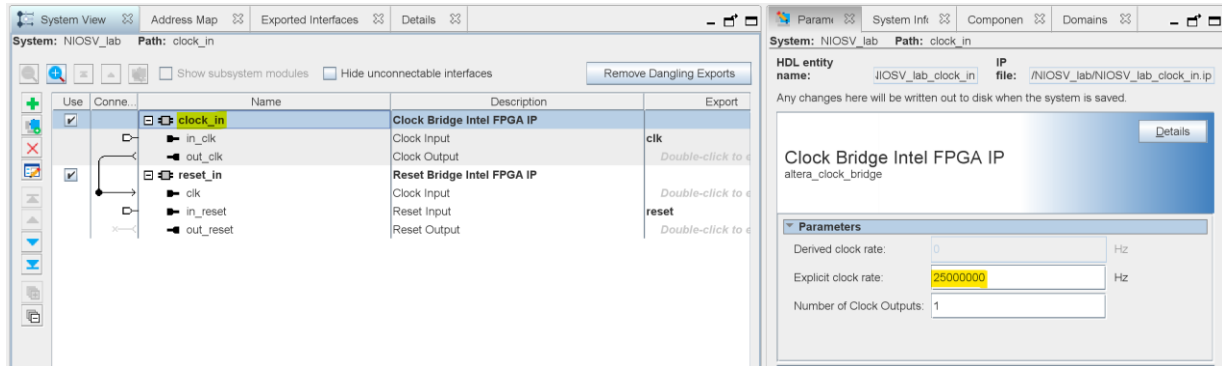


Figure 4-12 : Configure Clock Bridge

4.3.1.3 Configure the Reset Bridge

The Reset Bridge also has default settings that need to be changed. The Reset input on the board is tied to a push-button which goes low when pressed, we need to check the box for **Active low reset**.

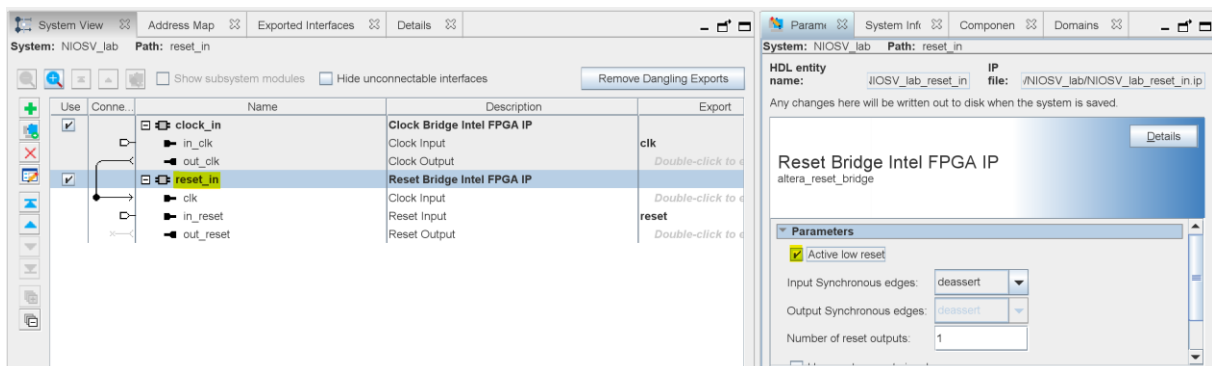


Figure 4-13 : Configure Reset Bridge

4.3.1.4 Add PLL IP

Since the input clock frequency of 25MHz is too slow, we need a Phase-Locked Loop (PLL) IP to multiply the frequency up to 100MHz (as example).

From the IP Catalog panel on the left side, expand the menus for the **Basic Functions** → **Clocks; PLLs and Resets** → **PLL** (or type *pll* in the search field) and double click on **IOPLL Intel FPGA IP**.

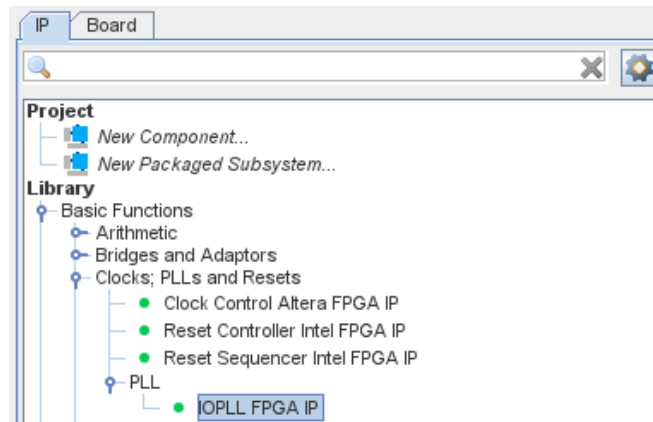


Figure 4-14 : PLL IP

A PLL (phase lock loop) provides flexibility with clock settings. It has a clock input and multiple clock outputs which can be set to different frequencies and phases.

In the **PLL** tab of the **IOPLL FPGA IP** settings window, do:

- Change the **Reference Clock Frequency** to 25.0 MHz to match our input clock,
- Uncheck the Enable locked output port,
- Keep the **Number Of Clocks** set to **1**,
- For **outclk0**, keep the default **100.0MHz** setting,

Change the HDL entity name to *iopll_0*.

- Click the **Finish** button.

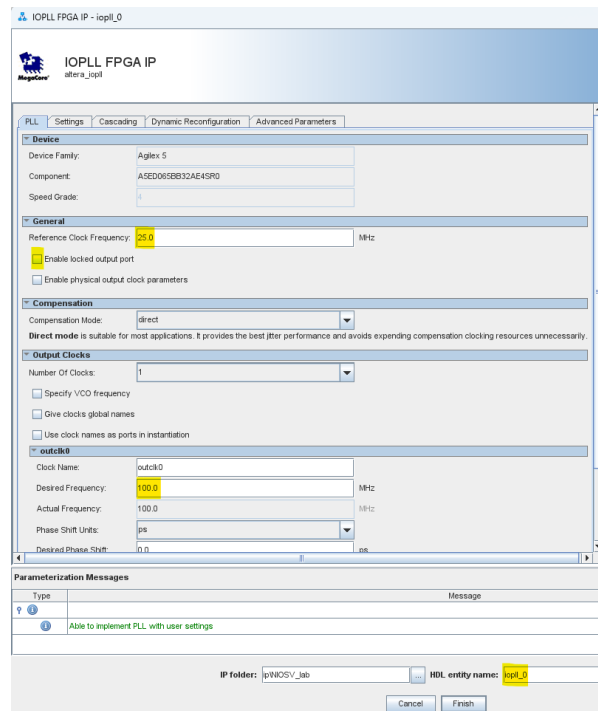


Figure 4-15 : PLL settings

By setting the reference clock frequency to 25MHz and matching it to the **clock_in** bridge IP, it gives us an input clock definition. Thus, it is not necessary to create an SDC constraint for it. Quartus uses this setting and the **outclk0** as clocks in the design.

4.3.1.5 Add Agilex Reset Release IP

The Agilex FPGA families require a coordinated release from the reset state after configuration is complete. This IP accomplishes this task.

In the **IP** pane, type **release** in the search field.

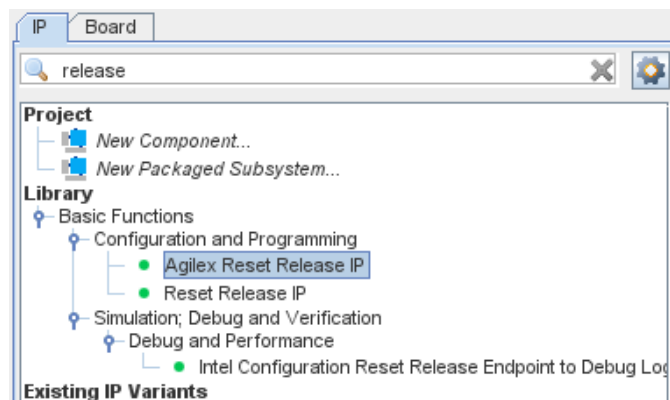


Figure 4-16 : Add Agilex Reset Release IP

Double-click Agilex Reset Release IP.

Check the **Reset Interface** radio button.

Change the HDL entity name to *reset_release*.

Click **Finish**.

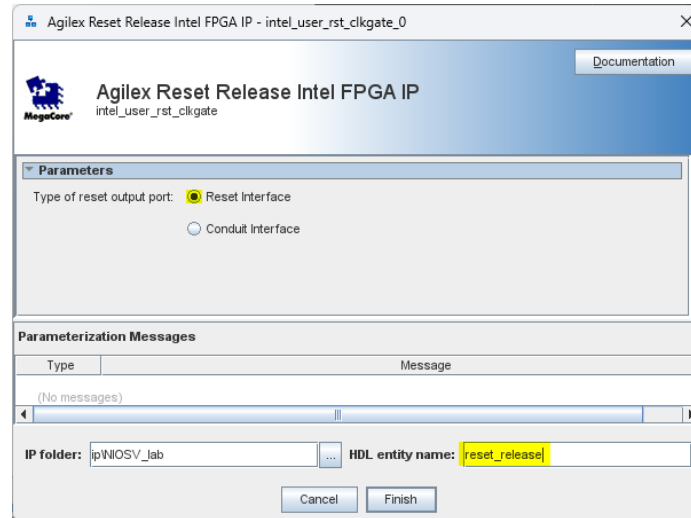


Figure 4-17 : Reset Release IP settings

4.3.1.6 Add Nios V/m IP

The Nios V CPU comes in 3 variants, /c (compact microcontroller), /m (microcontroller), and /g (general purpose processor). More details about these variants can be found [here](#). For the purposes of this lab, we will be using the /m variant.

In the IP search box, type **nios** and double-click **Nios V/m Microcontroller Intel FPGA IP**.

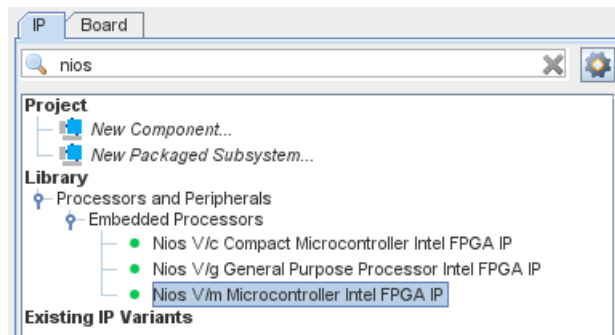


Figure 4-18 : Add Nios V/m IP

Change the HDL entity name to *niosv_m*. Click Finish.

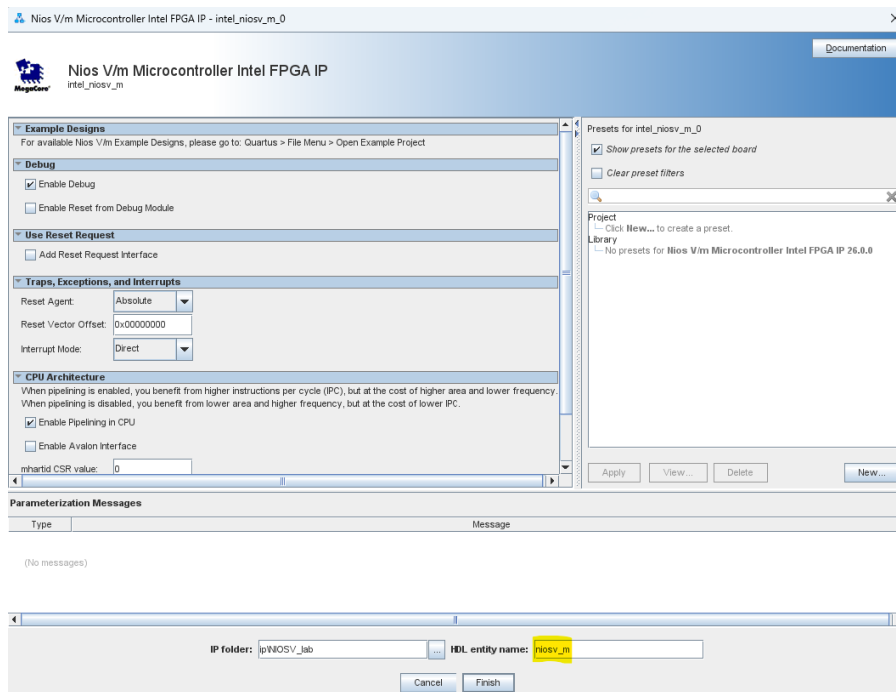


Figure 4-19 : Add Nios V/m settings

4.3.1.7 Add OnChip Memory IP

The Nios V needs memory to run its code from. In this lab, we will use a 160KB onchip SRAM for this purpose. For NIOS V, Intel recommends On-Chip Memory II IP.

In the IP search field, enter **onchip**.

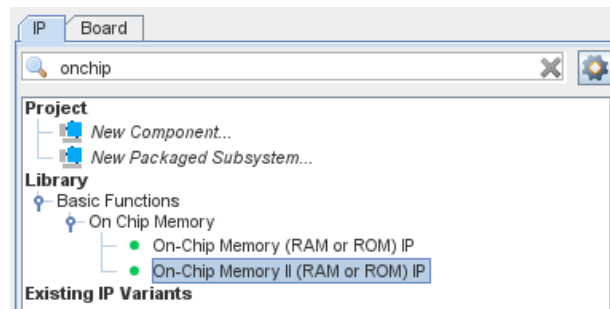


Figure 4-20 : Add on-chip SRAM IP

Double-click the **On-Chip Memory II IP**.

In the On-Chip Memory II wizard, do:

- Change the Total Memory Size to *163480*,
- Change the HDL entity name to *onchip_memory*,
- Check the **Initialize Memory Content** check box in the Memory Initialization section

- Check the **Enable non-default initialization file** check box. Use the default filename.
- Click **Finish**.

On-Chip Memory II (RAM or ROM) IP - intel_onchip_memory_0

On-Chip Memory II (RAM or ROM) IP
intel_onchip_memory

Size

S1 Data width: 32

Total memory size: 163480 bytes

Read latency

☐ Enable Level 1 output register for S1

☐ Enable Level 2 output register for S1

Read latency for S1 = 1.

ROM/RAM Memory Protection

Reset Request: Enabled

ECC

Extend the data width to support ECC bits for Tightly Coupled Memories: Disabled

Memory initialization

☒ Initialize memory content

☒ Enable non-default initialization file

Type the filename (e.g. my_ram.hex) or select the hex file using the file browser button.

User created initialization file: onchip_mem.hex

Parameterization Messages

Type	Message
(No messages)	

IP folder: ipNiosV_lab HDL entity name: onchip_memory

Cancel Finish

Figure 4-21 : Add on-chip SRAM settings

4.3.1.8 Add Lightweight UART IP

The Lightweight UART allows the user to interact with the C code using **stdio** operations without requiring an external UART device. In this lab, we will use it to display messages on a Putty or Tera term terminal emulator.

In the IP search field, type **uart**.

Double-click the **Lightweight UART IP**.

Keep all its default values.

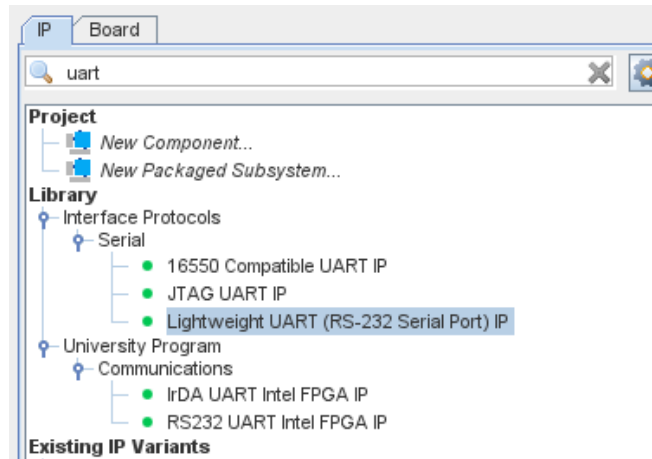


Figure 4-22 : Add JTAG UART IP

Change the HDL entity name to *lw_uart*. Click Finish.

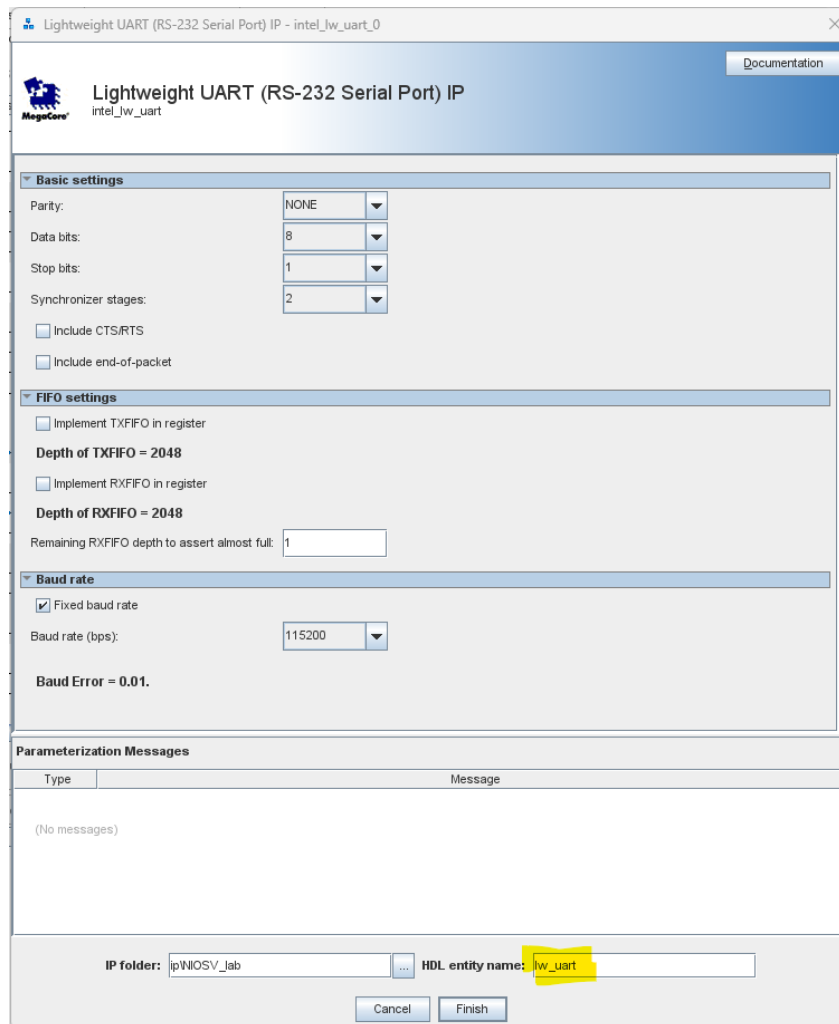


Figure 4-23 : Add JTAG UART settings

4.3.1.9 Add System ID IP

The system ID is a 32-bit register that can be used to give a design its unique identification such as hardware version.

In the IP search field, type **system**.

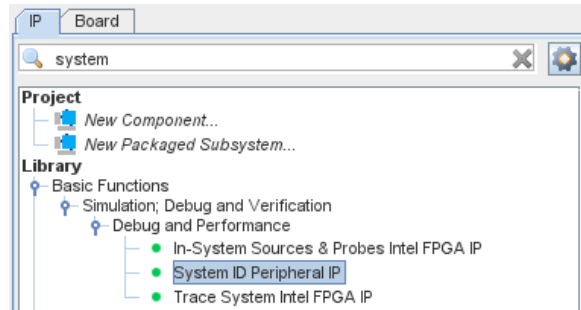


Figure 4-24 : Add System ID IP

Double-click the System ID Peripheral IP.

Change the 32-bit System ID field to *0x12345678*

Change the **HDL entity name** to *sysid*. Click Finish.

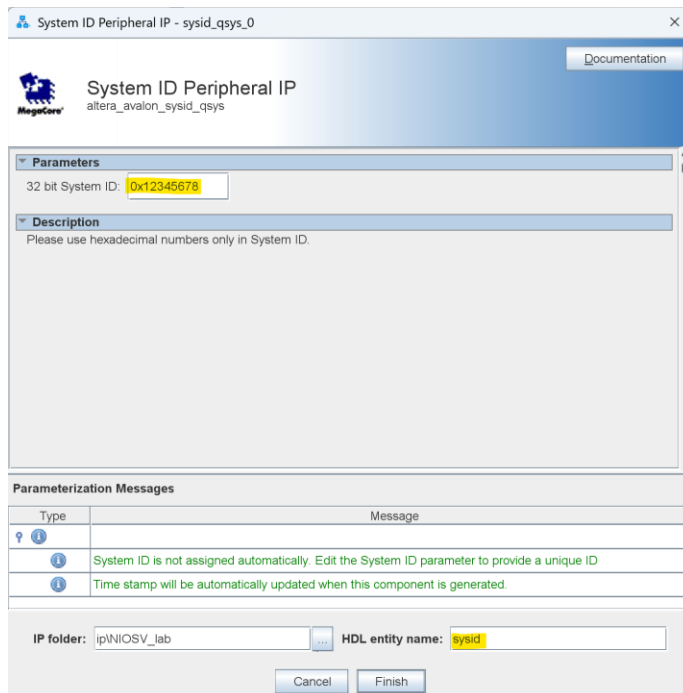


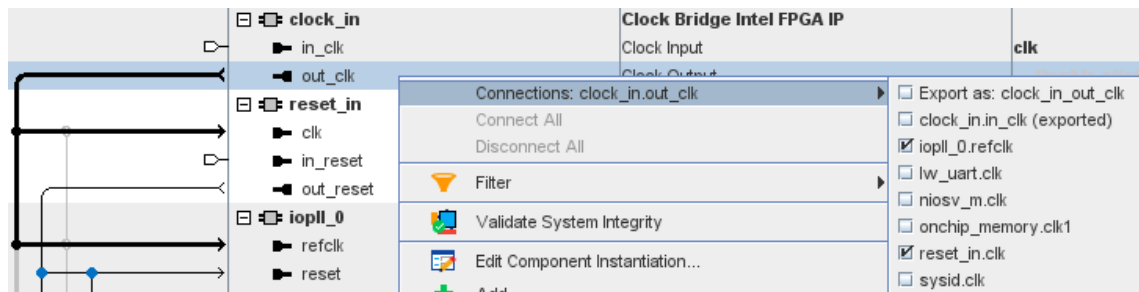
Figure 4-25 : Add System ID settings

At this point, you will see many errors and warnings in the bottom console indicating that various ports are not connected, and the memory addresses are not correct. Ignore them for now, as we will address them when we make connections.

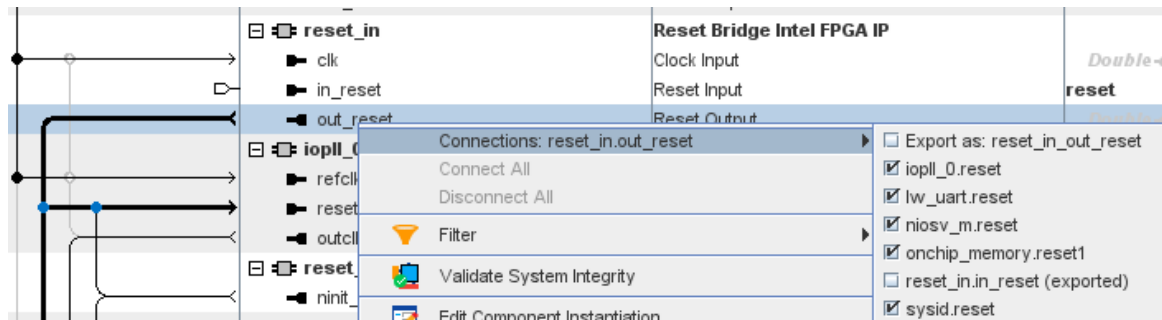
4.3.1.10 Connect the IP blocks and assign addresses

In the System View tab, make the following connections by clicking the intersection points. It is also possible to make the connections by right clicking and using the connections option shown in the images below.

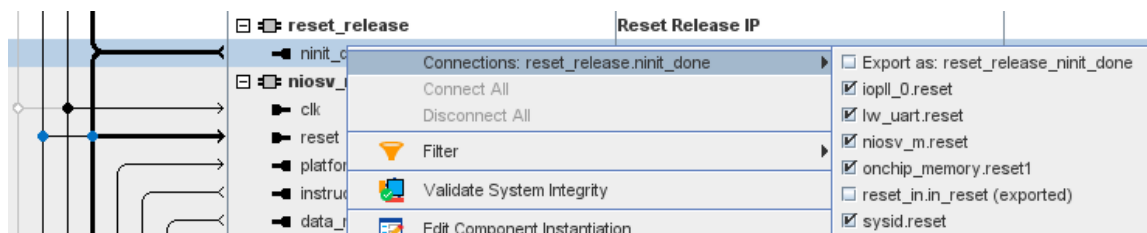
- Connect clock_in/out_clk to reset_in/clock and iopl0/refclk.



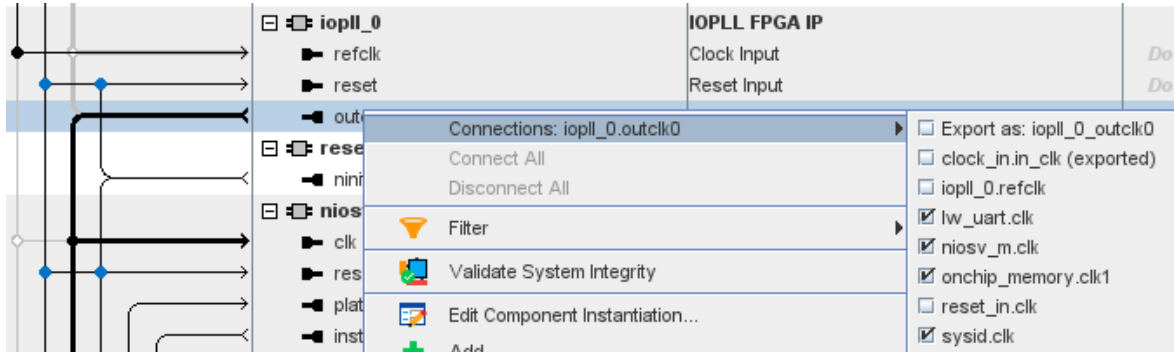
- Connect reset_in/out_reset to iopl0/reset, lw_uart/reset, niosv_m/reset, onchip_memory/reset1, and sysid/reset.



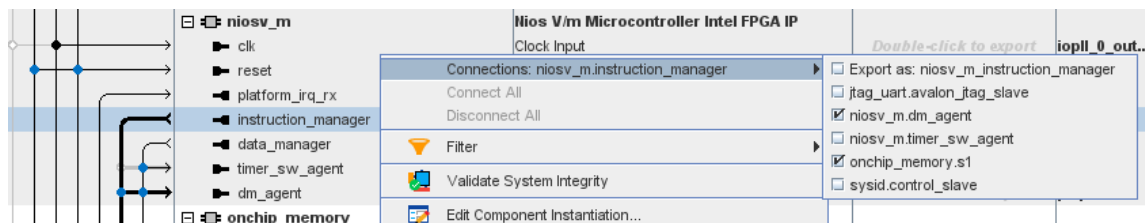
- Connect reset_release/ninit_done to iopl0/reset, niosv_m/reset, onchip_memory/reset1, lw_uart/reset, and sysid/reset.



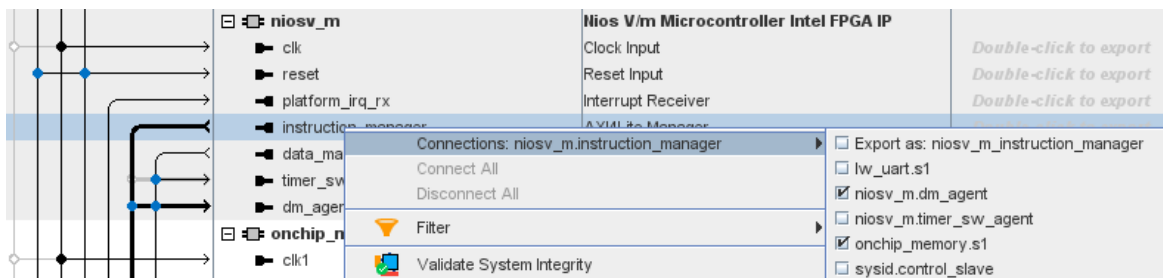
- Connect iopll_0/outclk0 to niosv_m/clk, onchip_memory/clk1, lw_uart/clk, and sysid/clk.



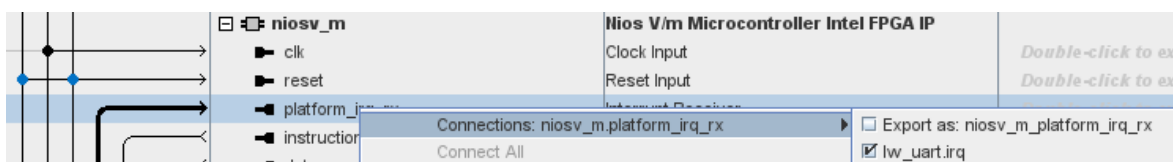
- Connect **niosv_m/instruction_manager** to **onchip_memory/s1**. This allows instruction execution from the onchip RAM.



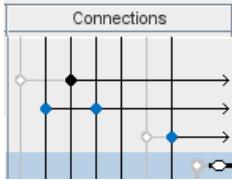
- Connect **niosv_m/data_manager** to **onchip_memory/s1**. This allows data fetch on from the onchip RAM.
- Connect niosv_m/data_manager to jtag_uart/avalon_lw_slave, and sysid/control_slave.



- Connect the niosv_m/platform_irq_rx to lw_uart/irq.



- Export the lw_uart signals. Enter uart in the export column of the external_connection field.

Connections	Name	Description	Export
	lw_uart	Lightweight UART (RS-232 Serial Port) IP	
	clk	Clock Input	<i>Double-click to export</i>
	reset	Reset Input	<i>Double-click to export</i>
	s1	Avalon Memory Mapped Agent	<i>Double-click to export</i>
	external_connection	Conduit	uart

From Platform Designer menu bar, execute **System → Assign Base Addresses**.

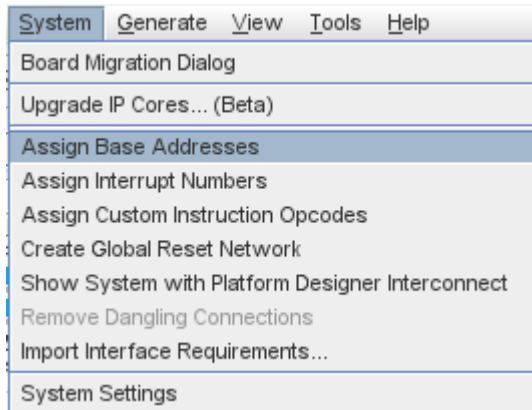


Figure 4-26 : Assign Base Addresses

When the **Sync System Infos** window completes, make sure it is successful with no errors. Click **Close**.

Ensure that the **onchip_memory** got assigned starting address **0x0000_00000**. This is where the reset vector will be.

At this point, there should not be any connectivity errors. See figure below.



Figure 4-27 : System Connections

After Addresses are assigned, the Nios-V module needs to be re-visited to assign the Reset Vector to the proper agent.

- Double-click the **niosv_m** module to reveal its parameters page.
- Change the **Reset Agent** setting in the **Vectors** category to **onchip_memory.s1**, thus setting it to point to the onchip RAM.

Figure 4-28 : Nios-V Reset Vector setting

Review the message window to make sure there are no errors.

4.3.2 Generate HDL for Platform Designer system

This stage generates RTL files that Quartus can use to compile the design.

- In the Platform Designer window, click the **Generate HDL ...** button in the lower right-hand corner to extract HDL code from the IP blocks added.
- In the Generation pop-up window, set the **Create HDL design files for synthesis** to **Verilog**.
- Check the box for Create block symbol file (.bsf) and keep all other default settings.
- Click the **Generate** button.

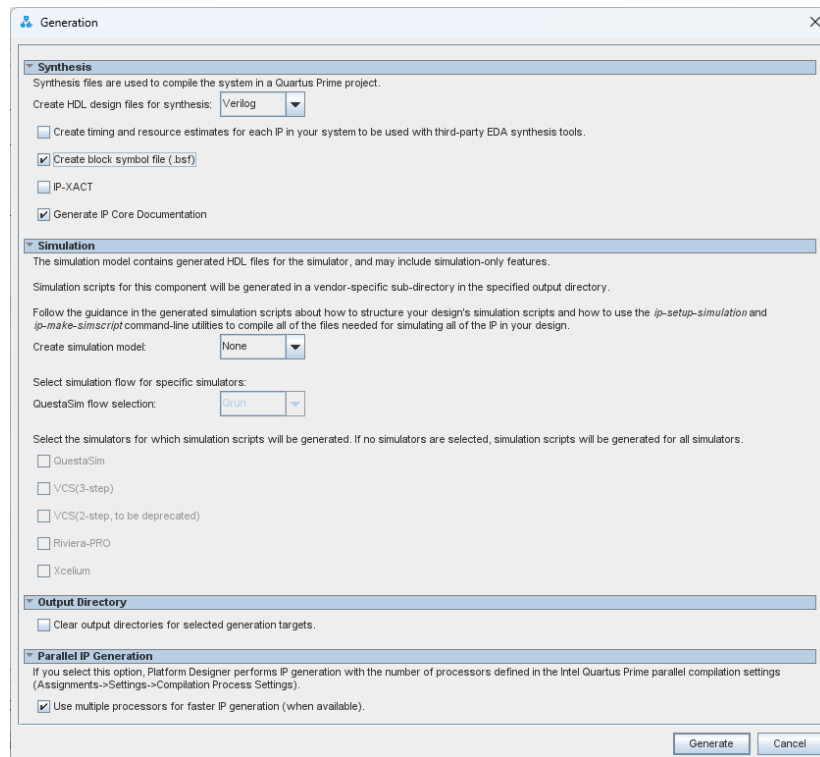


Figure 4-29 : HDL Generation

- When prompted to save, click **Yes**.
- The **Generate** window should complete successfully, with no errors. Click **Close**.
- Minimize the Platform Designer Window since the next part of the Lab will be done in Quartus.

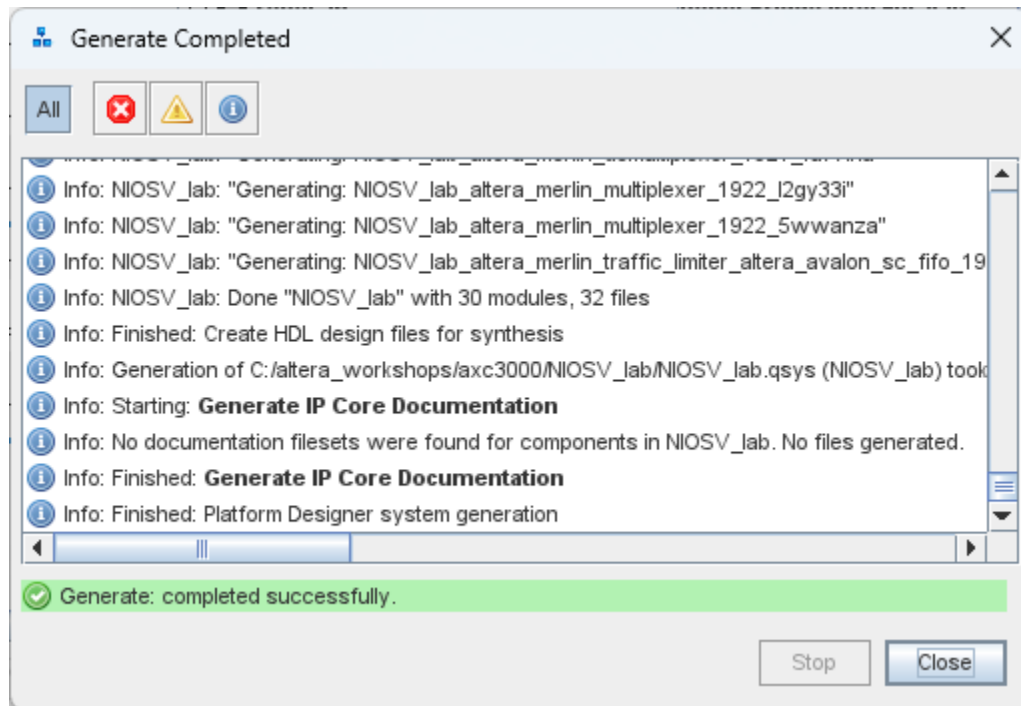


Figure 4-30 : Generate Complete

4.4 Complete the Quartus Design

During the creation of the new project, the top-level design entity is named NIOSV_lab . Therefore, the file **NIOSV_lab.qsys** is automatically the top-level design entity. Verify the assignment of the top-level design entity in the Project Navigator **Files** tab.

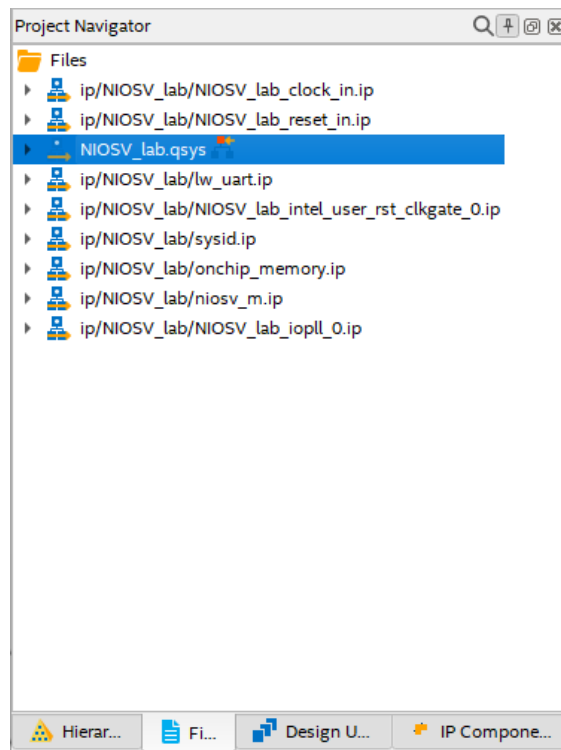


Figure 4-31 : Top-Level selection

The compile process consists of the steps shown below:

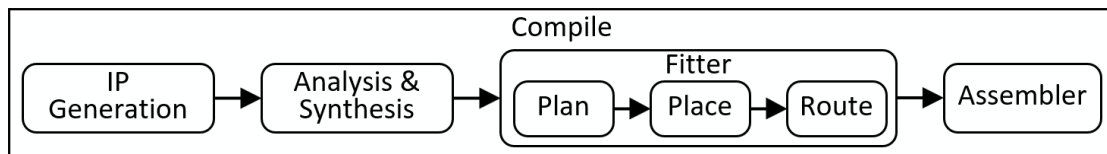


Figure 4-32 : Compile Process

We will be compiling the design in stages.

4.4.1 IP Generation

The IP Generation step makes sure that HDL code exists for all IP blocks. If not, it will generate them.

Click the IP Generation Blue arrow in the **Compilation Flow** window.

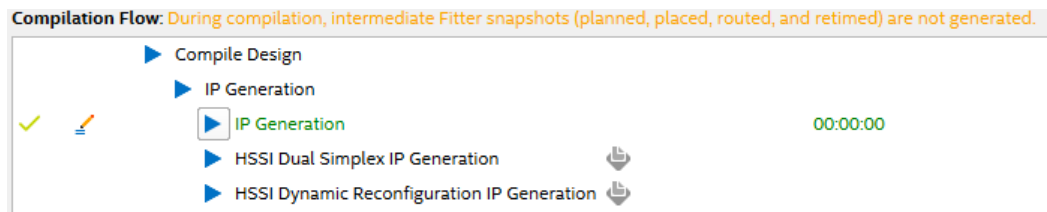


Figure 4-33 : IP Generation step

There should not be any errors generated by this step.

4.4.2 Analysis and Synthesis

Analysis and Synthesis, is the step where the design files are converted into a netlist that Quartus can use in later steps. Quartus also checks the legality of the connections. The netlist is used by Quartus so that IO constraints can be assigned.

This step creates a database in Quartus which allows us to assign I/O and other constraints. On the Quartus menu bar, select **Processing-> Start -> Start Analysis and Synthesis**, or click the icon.

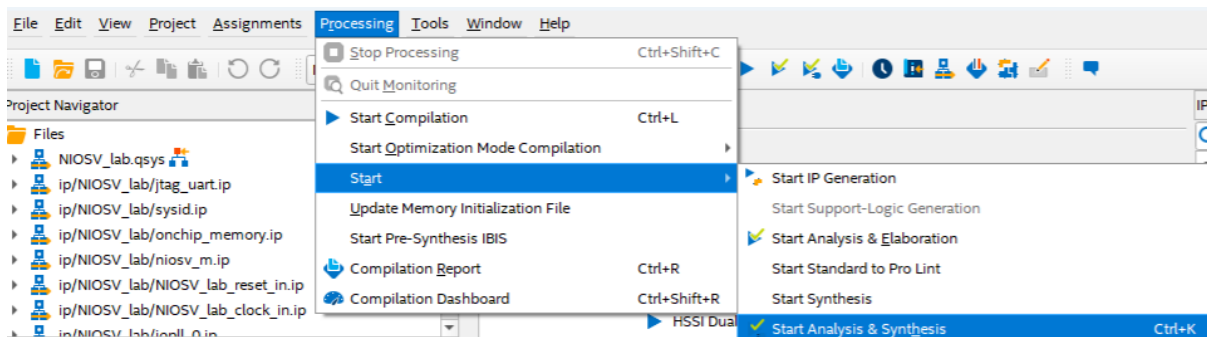


Figure 4-34 : Run Analysis & Synthesis

Upon completion, there should be green check marks to the left of the Analysis & Synthesis steps.

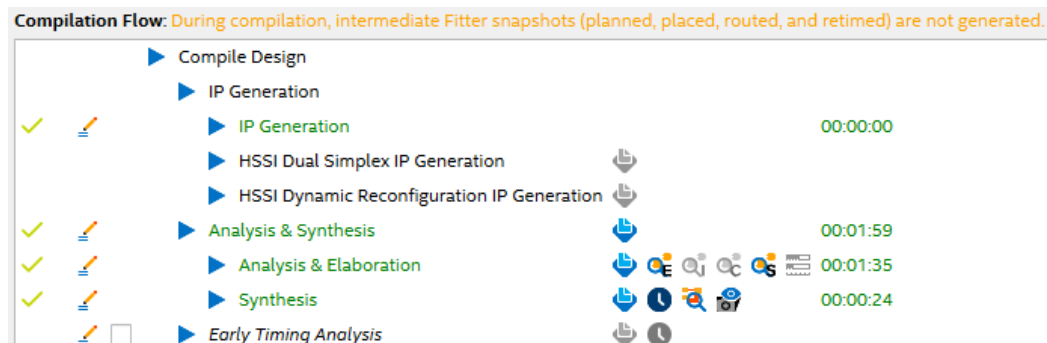


Figure 4-35 : Successful Analysis & Synthesis steps

4.4.3 Pin Assignment

Pin Assignments are needed so the design ports are assigned to the correct pins on the board. Without pin assignments, Quartus will use default pin assignments which will not correspond to the board pinout, and thus cause issues.

4.4.3.1 Open Pin Planner

Open the **Pin Planner** by clicking on  symbol on the toolbar, or **Assignments** → **Pin Planner**

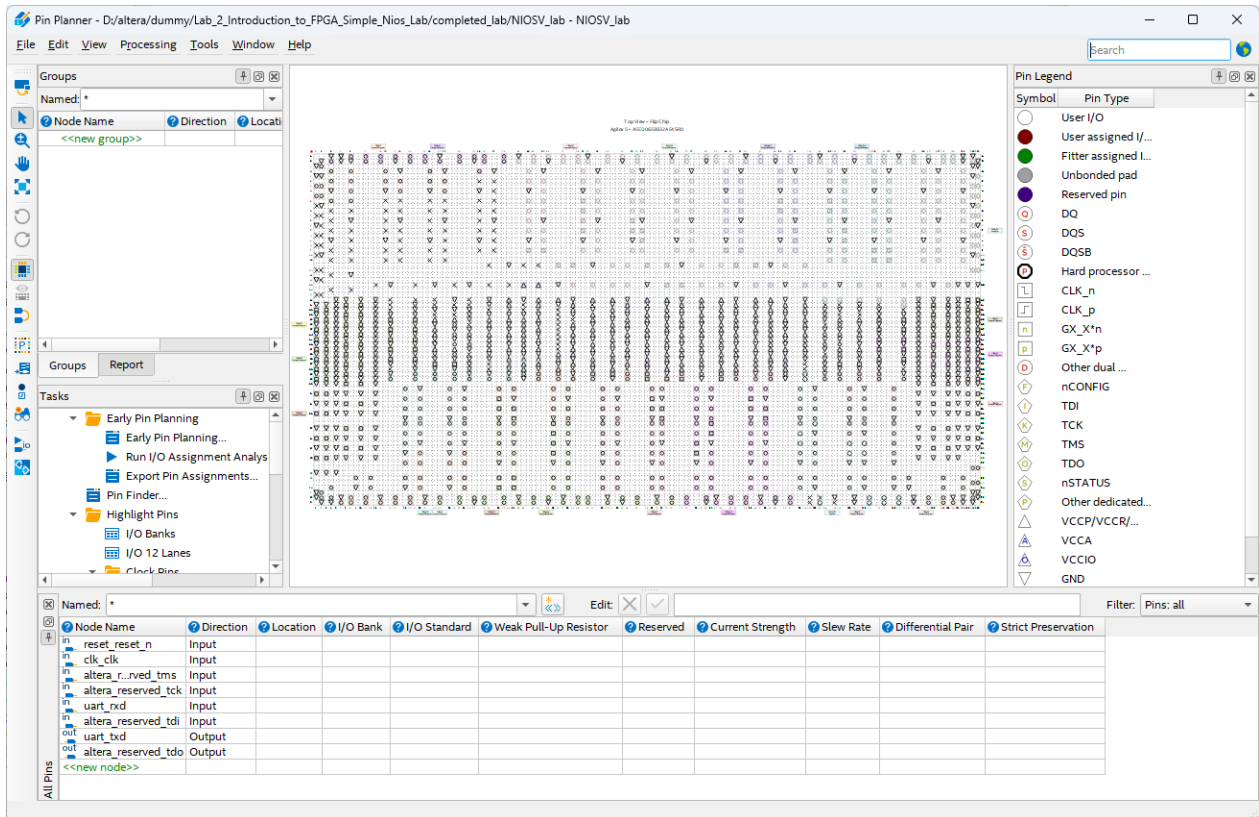


Figure 4-36 : Pin Planner

4.4.3.2 Assign Pins

The bottom section has a table which lists the design nodes that can be assigned to pins. Double-click the **Location** cell for node **clk_clk** and type **A7**.The “**PIN_**” prefix gets added by Quartus. Set its **I/O Standard** to **1.3-V LVC MOS**.

Double-click the **Location** cell for node **reset_reset_n** and type **A12**.The “**PIN_**” prefix gets added by Quartus. Set its **I/O Standard** to **1.3-V LVC MOS**. This pin is connected to a push-button labeled **S2** on the board.

Double-click the **Location** cell for node **uart_rxd** and type **AG23**.The “**PIN_**” prefix gets added by Quartus. Set its **I/O Standard** to **3.3-V LVC MOS**. This pin is connected to a UART to USB communication device.

Double-click the **Location** cell for node **uart_txd** and type **AG24**. The “PIN_” prefix gets added by Quartus. Set its **I/O Standard** to **3.3-V LVCMOS**. This pin is connected to a UART to USB communication device.

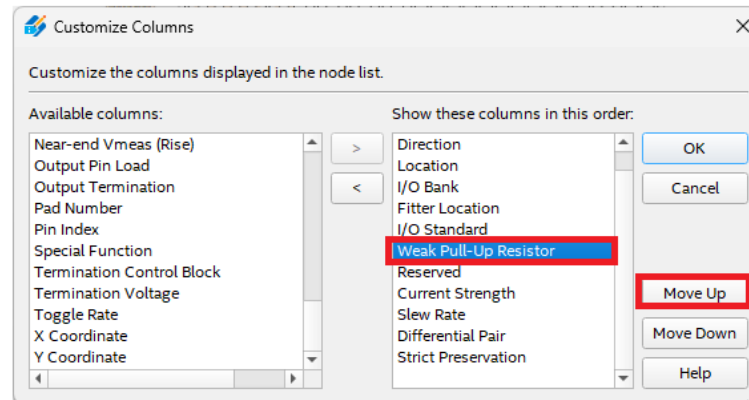
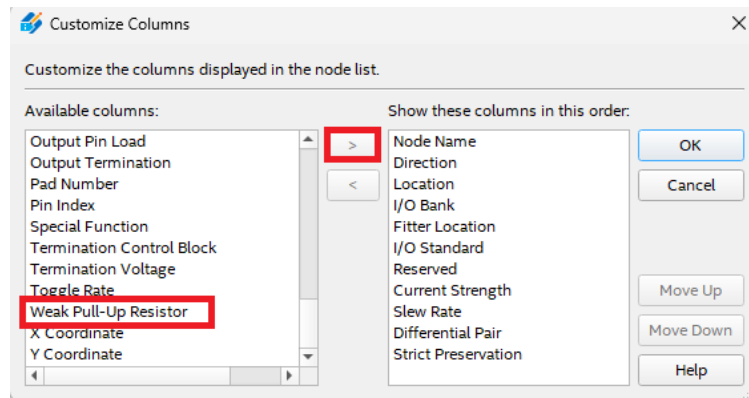
The JTAG pins are *reserved* and Quartus takes care of them. Leave them open.

? Node Name	? Direction	? Location	? I/O Bank	? I/O Standard
in reset_reset_n	Input	PIN_A12	3A_B	1.3-V LVCMOS
in clk_clk	Input	PIN_A7	3A_B	1.3-V LVCMOS
in altera_reserved_tms	Input			
in altera_reserved_tck	Input			
in uart_rxd	Input	PIN_AG23	5A	3.3-V LVCMOS
in altera_reserved_tdi	Input			
out uart_txd	Output	PIN_AG24	5A	3.3-V LVCMOS
out altera_reserved_tdo	Output			
<<new node>>				

Figure 4-37 : Pin Assignment

4.4.3.3 Customize Columns to add Weak Pull-up Resistor column

Right-click in any column heading of Pin Planner and **Check *Customize Columns***.box. This will open a dialog box. **Scroll down** in the **Available columns** region and **select *Weak Pull-up Resistor*** and **press** the **>** button. This adds the column in the far rightmost side of the table. You can move the **Weak Pull-up Resistor** column position by using the **Move Up** button. In the **Weak Pull-Up Resistor** column, **double-click** the box for the **uart_rxd** pin and set it to **on**.



Close the Pin Planner. The settings are automatically saved.

5 Software Project with AXC3000 board

Once the processor hardware system is ready, you can start building the software design using the **Ashling RiscFree™ IDE software for Intel FPGAs**.

RiscFree IDE is an Eclipse-based IDE that supports C and C++ software development. With the RiscFree environment, there are three main steps consisting of:

- Creating a C project environment with the hardware project imported,
- building an application C code
- compiling the code

Before the RiscFree IDE can be used, there are some set up items that need to be done first.

5.1 Create a New Software Project

To ensure an optimized compilation flow, it is recommended that you create a directory tree within your working directory. In your design project folder, create a folder named **software**. In the **software** folder, create two folders named **app** and **bsp**.



Figure 5-1 : Software folder tree structure

The **app** folder will have the C program and the **bsp** folder will have the Board Support Package data.

5.2 Generate the project BSP

The Board Support Package (BSP) is an integral part of the software development. It takes in the memory-mapped addresses assigned to memory and peripherals attached to the CPU bus and creates a Hardware Abstraction Layer (HAL) which contains the system definitions in a file called **system.h**, and collects the available API routines for the peripherals being used.

The BSP generation takes place in Platform Designer (PD). If PD is not already open with the NIOS-V system loaded, open it with **Tools → Platform Designer**. In the **Platform Designer system** entry box, navigate to **NIOSV_lab.qsys** and **select** it. Click **Open**.

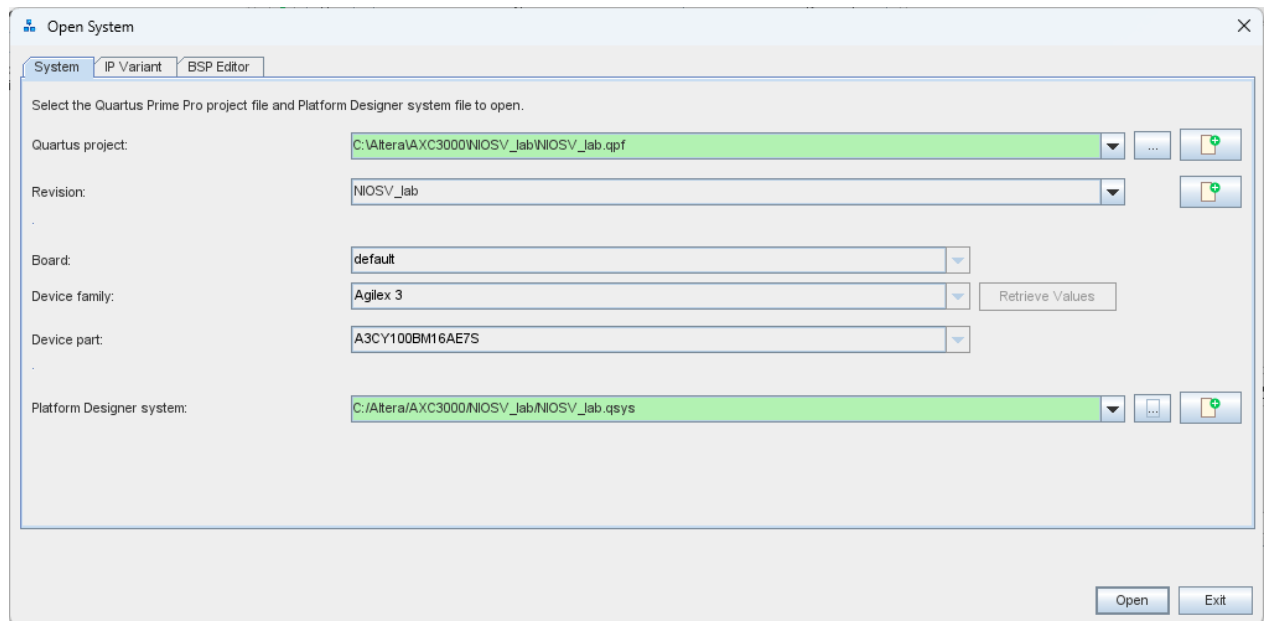


Figure 5-2 : Open System window from Platform Designer

Once the system is opened, you can **close** the dialog box.

In the Platform Designer window, go to **File → New BSP**.

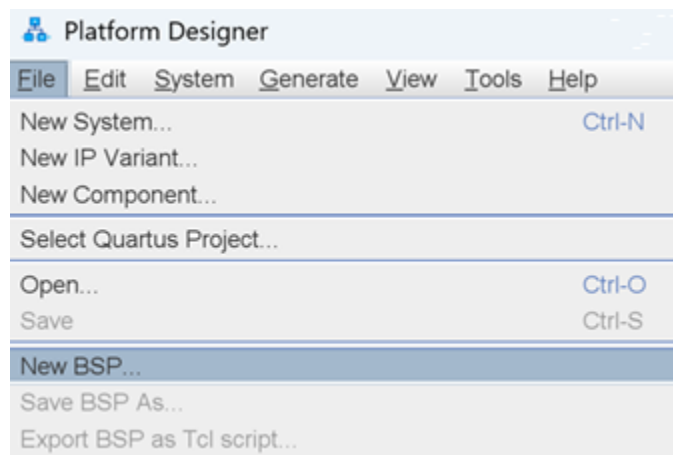


Figure 5-3 : New BSP

You will be asked to select the BSP name. Navigate to the **software → bsp** folder and use the file name **settings**. Click the **Create** button.

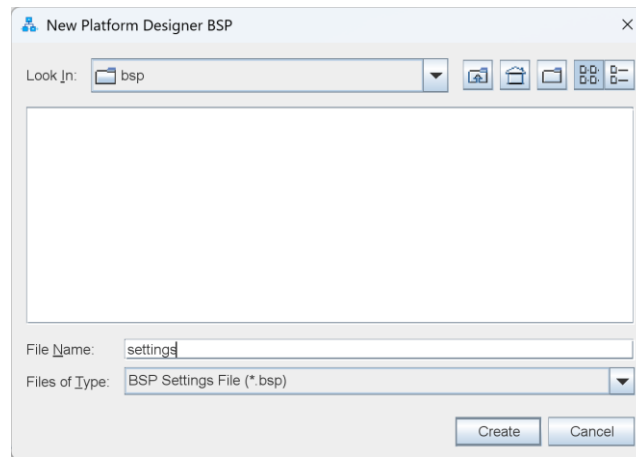


Figure 5-4 : BSP Settings file name

The next entry is to select the **System file**.

Navigate to the ***NIOSV_lab.qsys*** file and select it.

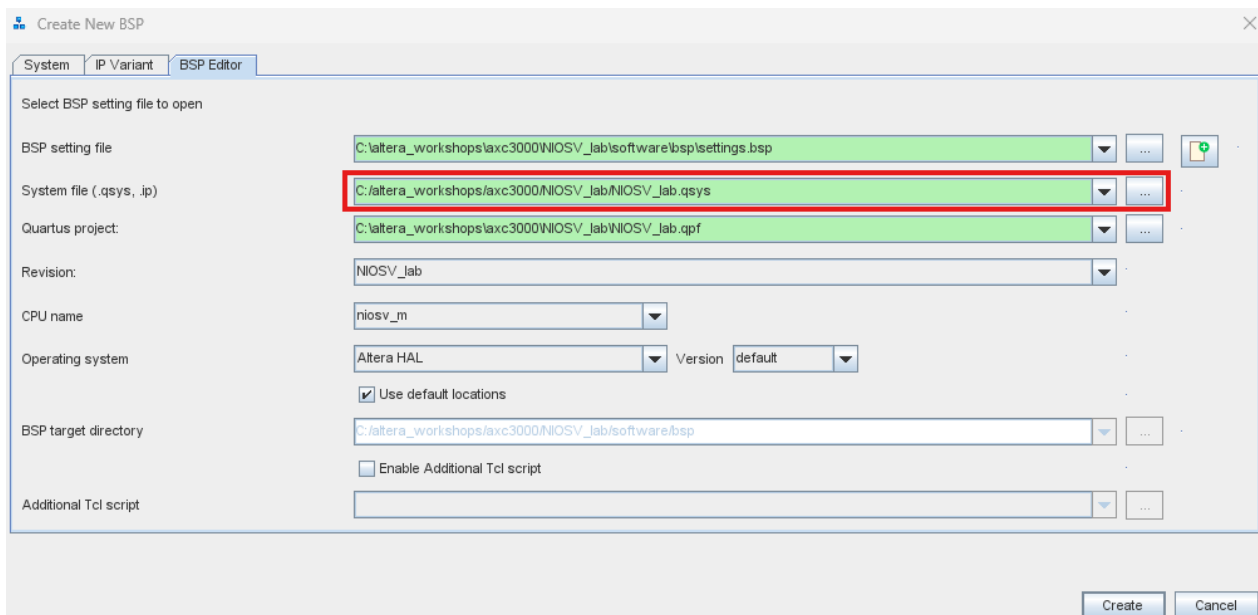


Figure 5-5 : Select System File name

Click **Create**.

When the BSP creation operation is done, the BSP Editor will display on the right-hand side of the Platform Designer.

Click the **Generate BSP** button. You may have to make that side of the window wider to see this button.

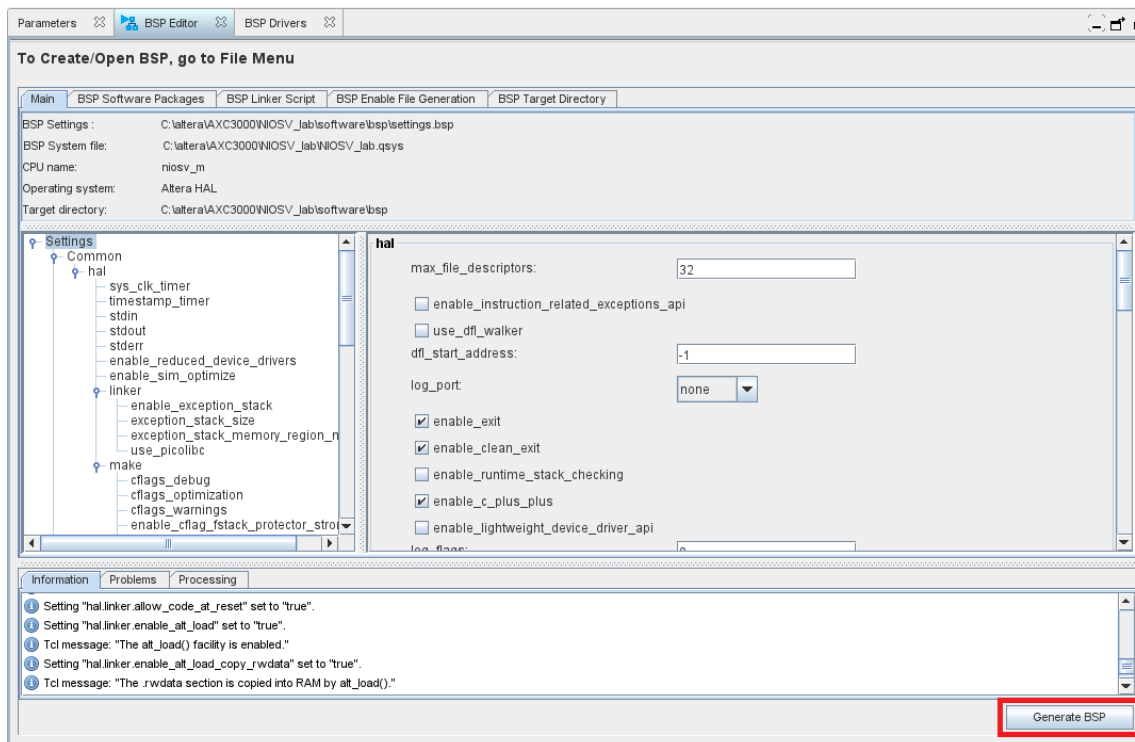


Figure 5-6 : Generate BSP

The BSP files are generated in the **software/bsp** folder.

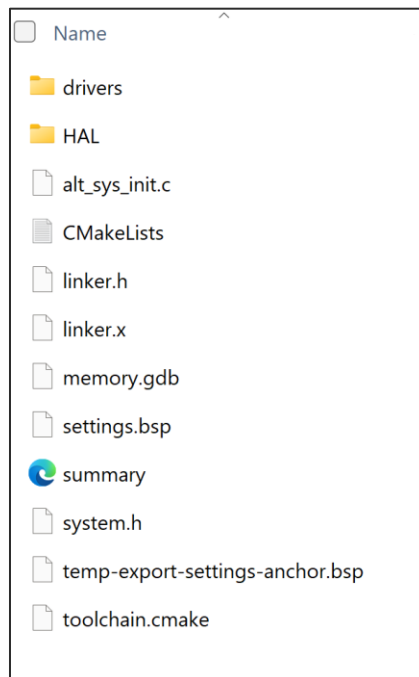


Figure 5-7 : BSP files

The **system.h** file describes Platform Designer peripherals. **alt_sys_init.c** has initialization drivers. **linker.h** describes the memory layout.

5.3 Building software project using Ashling RiscFree IDE for Intel FPGA

We already have a **software/app** folder.

- We will create a **HelloWorld.c** file and place in it.
- The application project needs to be created using the **settings.bsp** files created earlier. This creates more files needed by the Ashling RiscFree compiler.

5.3.1 Create the C code program

In the software directory **software/app**, create a C source file with code below and name it **HelloWorld.c**. For convenience and correctness, copy and paste the code below in Figure 57.

The BSP generation added some Hardware Abstraction Layer (HAL) API routines for you to use.

The **system.h** file has the address assigned to **SYSID_BASE** variable.

The `IORC_ALTERA_AVALON_SYSID_QSYS_ID` command is an API found in `altera_avalon_sysid_qsys.h`.

```
#include "system.h"
#include "stdio.h"
#include "alt_types.h"
#include <sys/alt_sys_init.h>
#include "altera_avalon_sysid_qsys.h"
#include "altera_avalon_sysid_qsys_regs.h"

alt_u32 temp_data;

int main()
{
    // Initialize all of the HAL drivers used in this design
    alt_sys_init();

    // Read and display the system ID value created in hardware
    temp_data = IORC_ALTERA_AVALON_SYSID_QSYS_ID(SYSID_BASE);
    printf("\r\nHello World! My System ID is: 0x%08lX\r\n", temp_data);

    while (1);

    return 0;
}
```

Figure 5-8 : Application C code

5.3.2 Create the Application Project

In addition to the BSP, the software project needs an application project which is created using a CMake flow. It uses the C code program, the BSP, and the HAL to generate an application environment used by the Ashling RiscFree IDE.

5.3.2.1 Open a Nios V Command Shell

Open the Nios V command shell from the Windows Start button -> All apps -> Altera 25.1.1.125 Pro Edition -> Nios V Command Shell (Quartus Prime Pro 25.1.1)

This will open a command shell window.

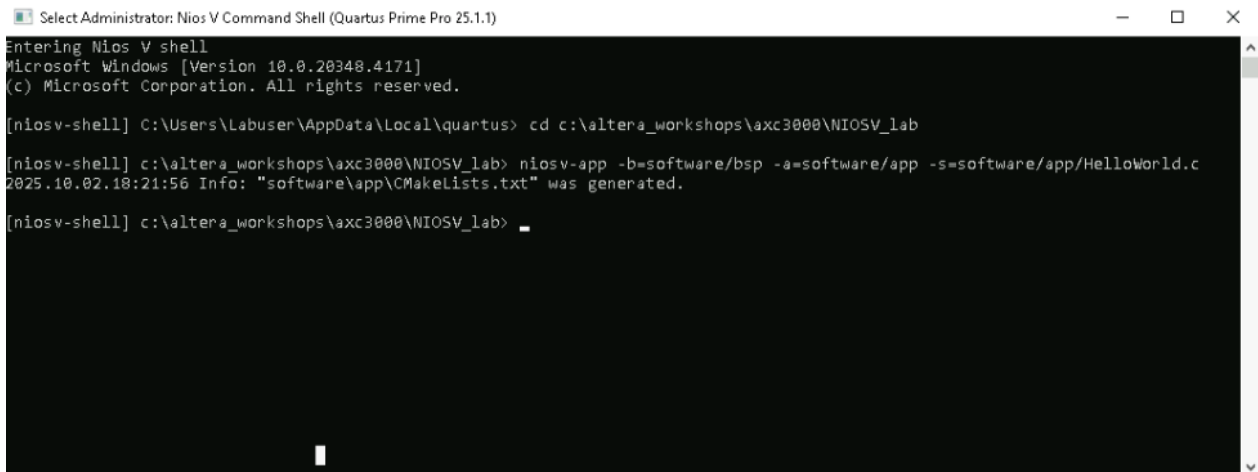
Navigate to the project folder C:\altera_workshops\axc3000\NIOSV_lab

You'll need to remove the existing local path before entering the project folder path. To do this, type `cd` in the command shell, followed by the full path to your project folder (see example in Figure 5-9).

5.3.2.2 Create Application

In the Nios V command shell, execute the command shown below, to generate the CMake application file **CMakeLists.txt**. This file is used by the Ashling RiscFree compiler.

```
niosv-app -b=software/bsp -a=software/app -s=software/app/HelloWorld.c
```



```
Select Administrator: Nios V Command Shell (Quartus Prime Pro 25.1.1)
Entering Nios V shell
Microsoft Windows [Version 10.0.20348.4171]
(c) Microsoft Corporation. All rights reserved.

[niosv-shell] C:\Users\Labuser\AppData\Local\quartus> cd c:\altera_workshops\axc3000\NIOSV_lab

[niosv-shell] c:\altera_workshops\axc3000\NIOSV_lab> niosv-app -b=software/bsp -a=software/app -s=software/app/HelloWorld.c
2025.10.02.18:21:56 Info: "software\app\CMakeLists.txt" was generated.

[niosv-shell] c:\altera_workshops\axc3000\NIOSV_lab> _
```

Figure 5-9 : App Generation with Nios V Command Shell

5.3.2.3 Use Ashling RiscFree IDE

Opening the **Ashling RiscFree IDE** can be done by typing **riscfree -data software** in the Nios V Command Shell. This points the RiscFree Workspace to the **software** folder.

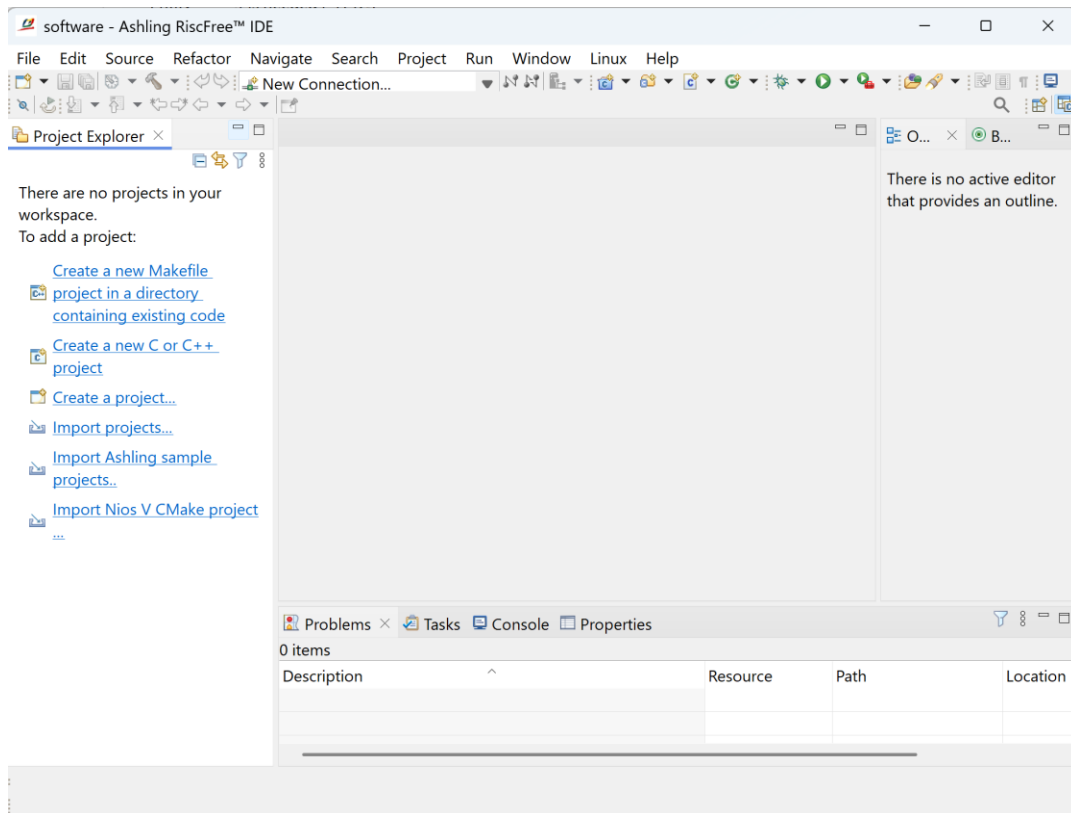
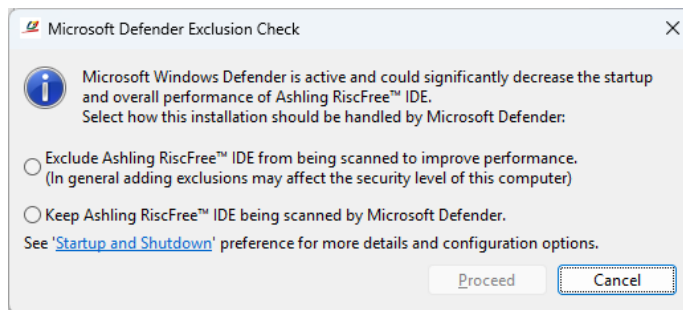


Figure 5-10 : Open Ashling RiscFree IDE

Click on **Create a project...** in the thumbnails.

Click **Cancel** if you see this pop up.



Expand the **C/C++** wizard and select the **C Project** item, then click **Next**.

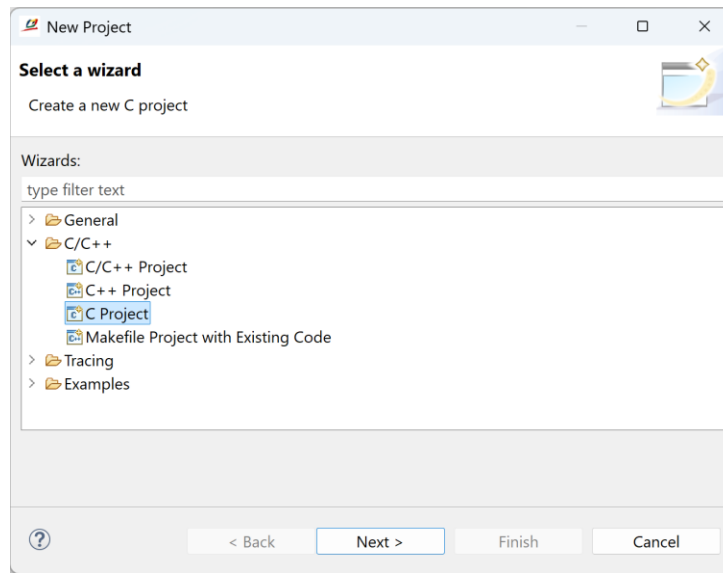


Figure 5-11 : Select C Project type

Expand the **CMake driven** Project type and select **Empty Project**. In the Toolchains pane, select **CMake driven**.

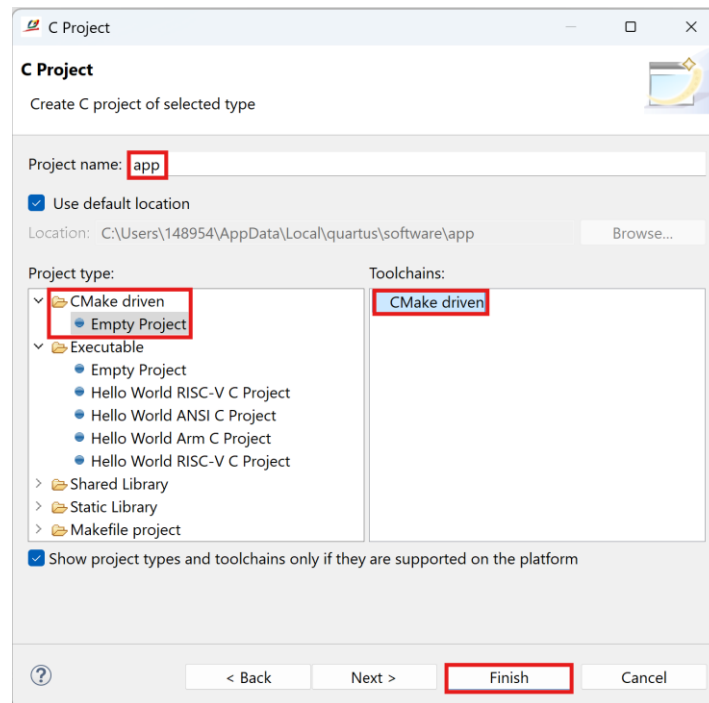


Figure 5-32 : Create CMake driven demo_app project

Type **app** for the Project name. This matches the pre-existing folder containing the application project.

Click **Finish**.

In the **Ashling RiscFree IDE**, you should see the **app** folder.

Right-click the **app** project and select **Build Project**.

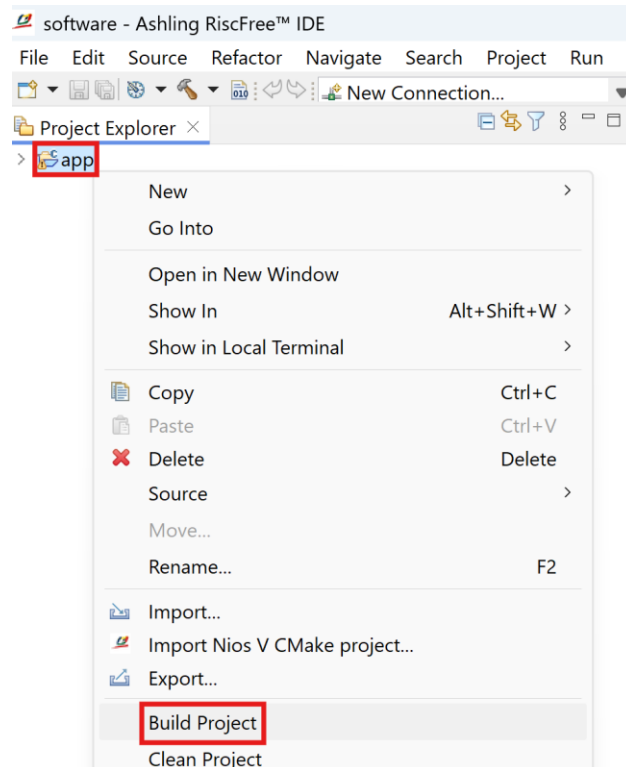


Figure 5-13 : Build Software Project

When the **Build Project** progress meter in the lower right corner reached 100%, always check the **Problems** messages window for error message.

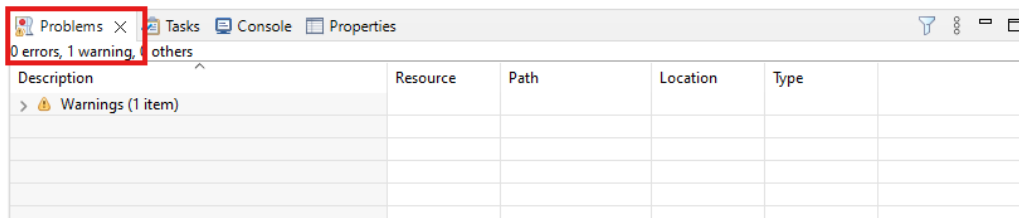


Figure 5-14 : Compiler Problems Window

5.3.3 Convert ELF code to HEX

The NIOS-V C code has been successfully compiled, but still needs to be converted into a .hex format for placement in the FPGA onchip RAM during Quartus compilation. Execute the command below in the Nios V Command Shell.

```
elf2hex software/app/build/Debug/app.elf -b 0x00000000 -w 32 -e 0x00027E97 --
record=4 -o onchip_mem.hex
```

5.4 Compile the Design in Quartus

The hardware project needs to be compiled with no errors in order to get a **.sof** configuration file. There are multiple steps performed in Quartus. These steps are the **Fitter** and **Assembler** steps. Ensure that the box to the left of the **Assembler** step is checked. This is the step that generates the **.sof** configuration file.

You can either re-start Compilation from the beginning, by clicking on  button on the toolbars, or **Processing** → **Start Compilation**, or resume to the **Fitter** and **Assembler** by clicking the blue arrow to the left of **Assembler**.

There should be no errors during compilation. If there are errors, they should be fixed before re-compiling. A green checkmark next to the Compile steps in the **Compilation Flow** window indicates that the compilation was successful.

As Quartus compiles the design, there will be a progress bar that shows the steps that it does and the checkmarks as the steps are completed. Notice the checkmarks as steps are completed. The time used for each step depends on the computer used such as the number of processors, amount of memory, and processor clock speed.

If you did a re-compile from the beginning, there will be a **Timing Analysis** Window that appears at the end, but the focus is on the compilation.

The on-chip memory is initialized with the software project compiled code in a HEX format. This will be included in the FPGA bitstream when the device is programmed and will be run when the Nios V processor is released from reset.

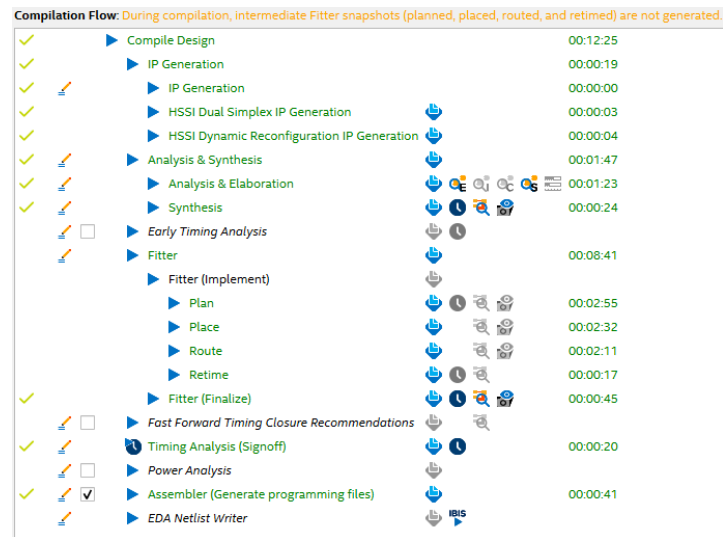


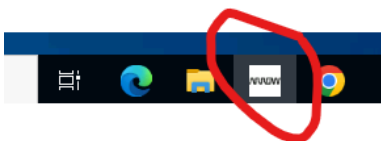
Figure 5-15 : Completed Compile flow

6 Validate the design Locally

The next step is to see if the design works. If you have built the lab using CloudLabs then complete section 6.1. If you have built the lab on a local PC then skip to section 6.2.

6.1 File Download

6.1.1 Click on the ArrowDTD icon in CloudLabs to launch the tool.

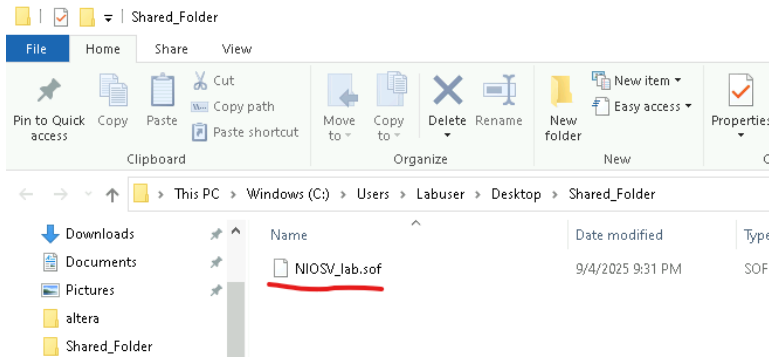


6.1.2 Open the **Shared Folder** utility in the CloudLab VM. Open a browser in your local PC. Navigate to the **URL** indicated. Use the provided username and password.

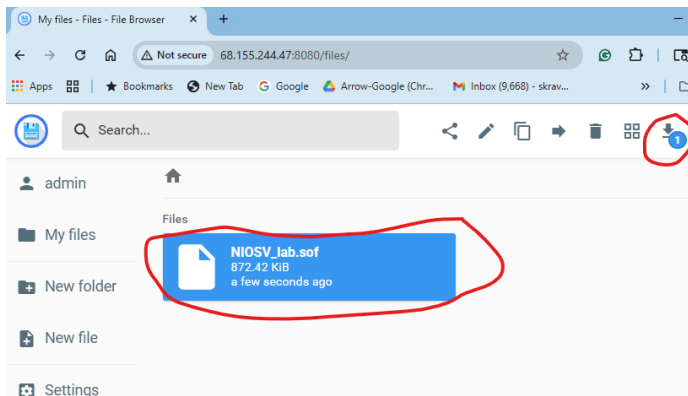


6.1.3 **Drag** and drop the following file into the **Shared** folder.

c:\altera_workshops\axc3000\My_First_Lab\output_files\NIOSV_lab.sof



6.1.4 This will be reflected in the **browser** on the local PC. **Download** the SOF file to a convenient location.



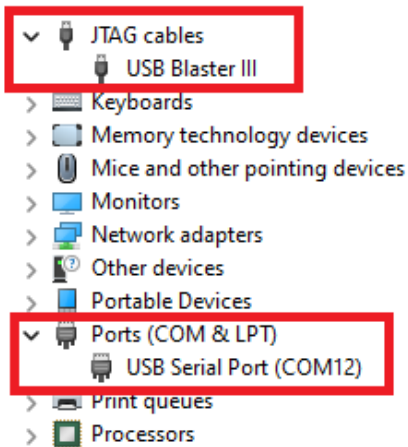
The rest of the instructions will be done on the **local computer**.

6.2 Program the FPGA

The next step is to program the design into the FPGA on the board. During the Assembler stage of the compilation, a programming file was generated.

6.2.1 Connect the AXC3000 to your PC using a USB C cable.

Since the Altera USB Blaster III should be already installed, the Window's Device Manager should display the following entries, highlighted in red (COM port number may differ depending on your PC). Use the COM port later when setting up the terminal emulator.



6.2.2 From the Start menu open Altera 25.1.1.125 Pro Edition --> Programmer.

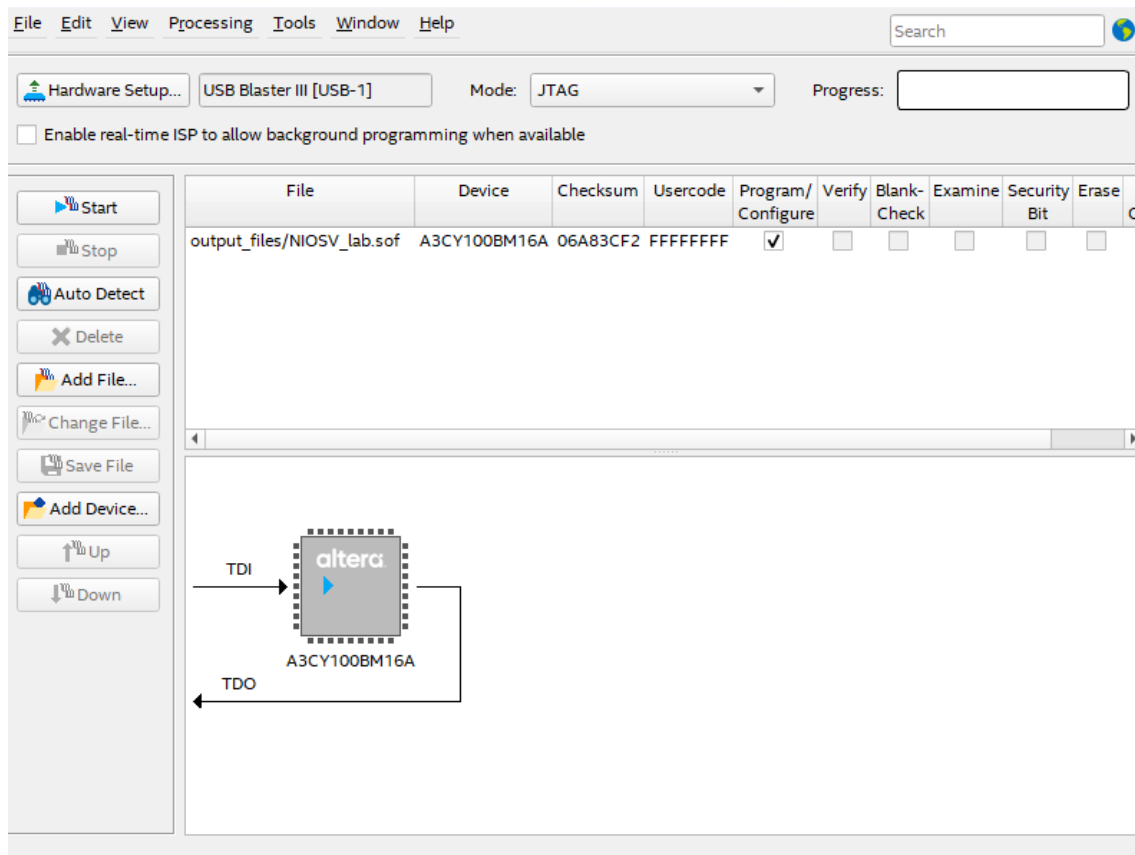


Figure 6-1 : Quartus Programmer

- 6.2.3 Click the **Hardware Setup** button and double-click on the **USB Blaster III**, then click **Close**.

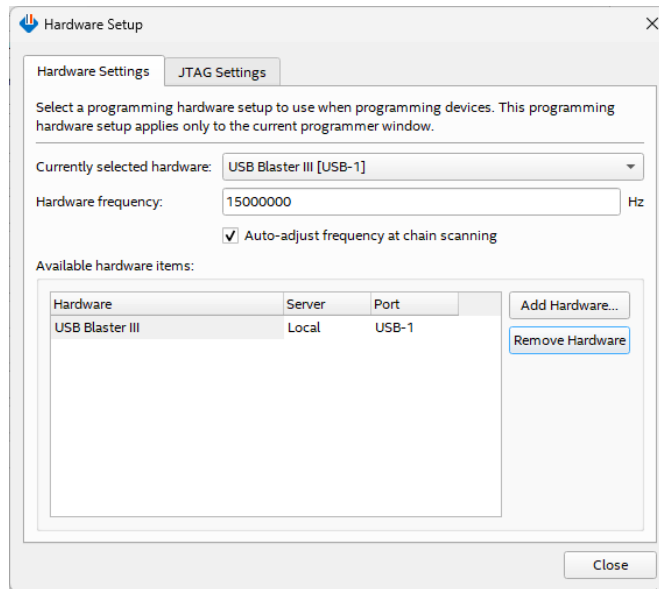


Figure 6-2 : USB Blaster III Hardware Setup

- 6.2.4 Ensure that the **USB Blaster III** is selected, and the mode is set to **JTAG**.
- 6.2.5 If the .sof file is not already selected, click on the **Change File** button and navigate to the **output_files/NIOSV_lab.sof**. If the file was downloaded from Cloudlabs then navigate to the **folder where the sof file was downloaded** and select the **.sof** file.

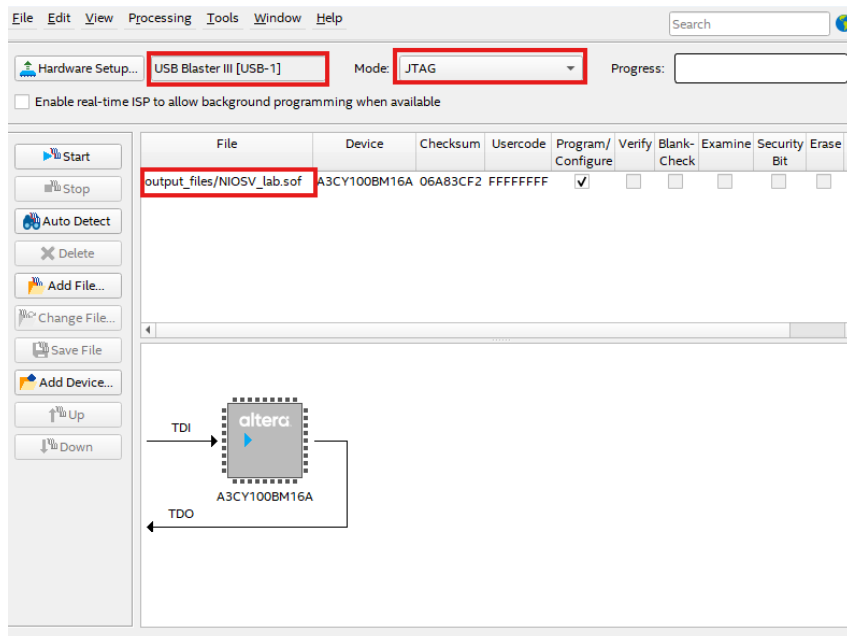


Figure 6-3 : USB Blaster III and .sof Selected

- 6.2.6 Select the .sof file then click the **Start** button to begin the programming operation and wait for the progress bar to hit 100%.

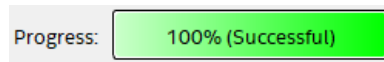


Figure 6-4 : Programmer progress bar

6.3 Open a Terminal Emulator

Open a terminal emulator (Tera term or Putty). Select the COM port shown earlier. Set the emulator to have 8 databits, 1 stop bit and no Parity bit(None). Set the baud rate to 115,200.

Observe the “**Hello World**” message printed.

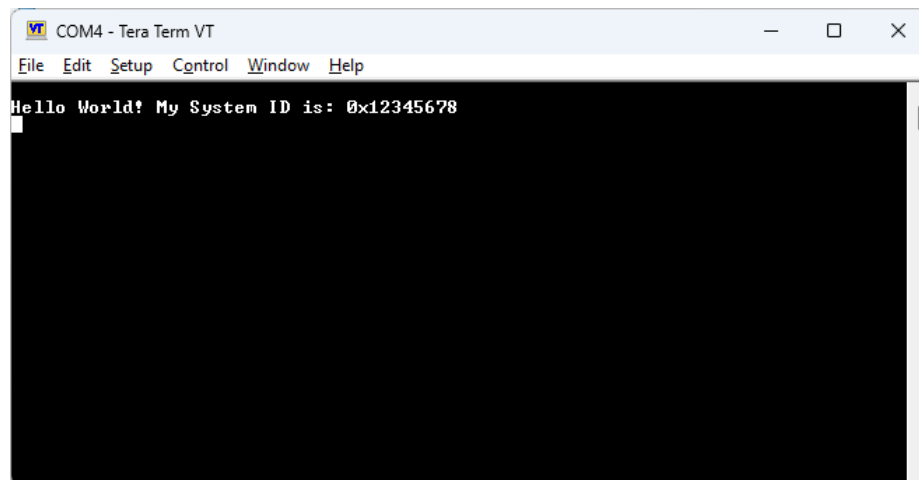


Figure 6-5 : Terminal Emulator

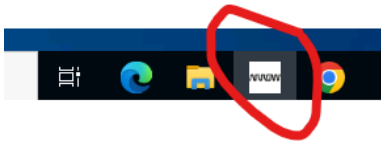
**CONGRATULATIONS! YOU HAVE SUCCESSFULLY COMPLETED
THE Nios V DESIGN LAB!**

7 Validate the Design Remotely

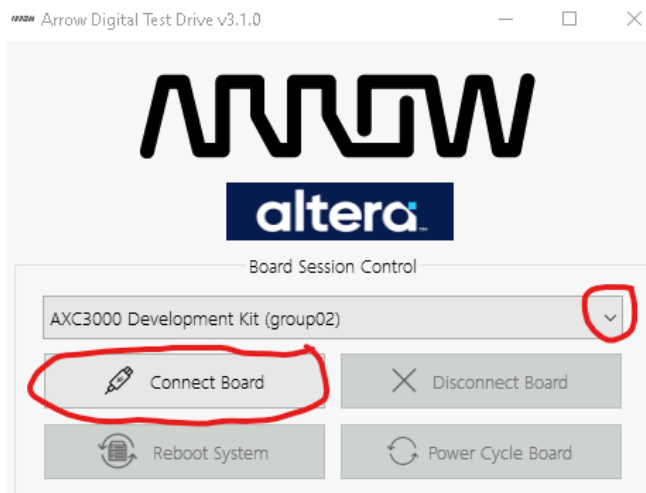
The following instructions require the user to be utilizing a CloudLabs Windows Virtual Machine.

7.1 Connect to the Board Farm

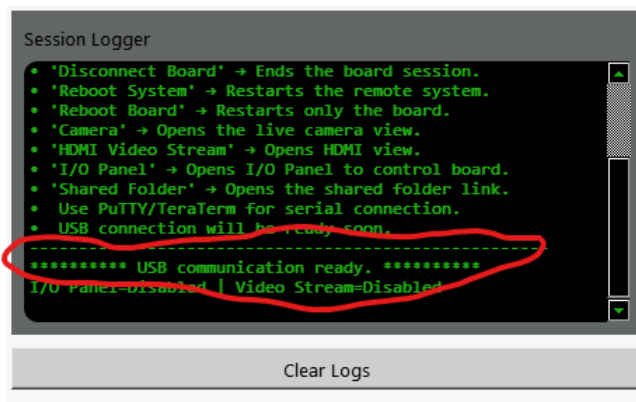
- 7.1.1 Connect to the remote AXC3000 Board using the ArrowDTD tool. Click on the ArrowDTD icon to launch the tool.



- 7.1.2 Select an available board from the dropdown menu and click connect board.



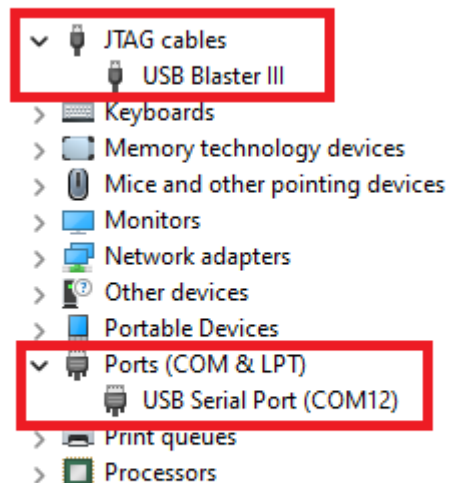
Wait until the board is ready.



7.1.3 Open the Camera View of the board.



7.1.4 Window's Device Manager should display the following entries, highlighted in red (COM port number may differ depending on your PC). Use the COM port later when setting up the terminal emulator.



7.2 Program the FPGA

7.2.1 Open the Quartus Prime Programmer

On the Quartus Prime Pro, click **Tools → Programmer**, or click on the Programmer icon  in the tool bar. The **Hardware Setup** field might show **No Hardware**.

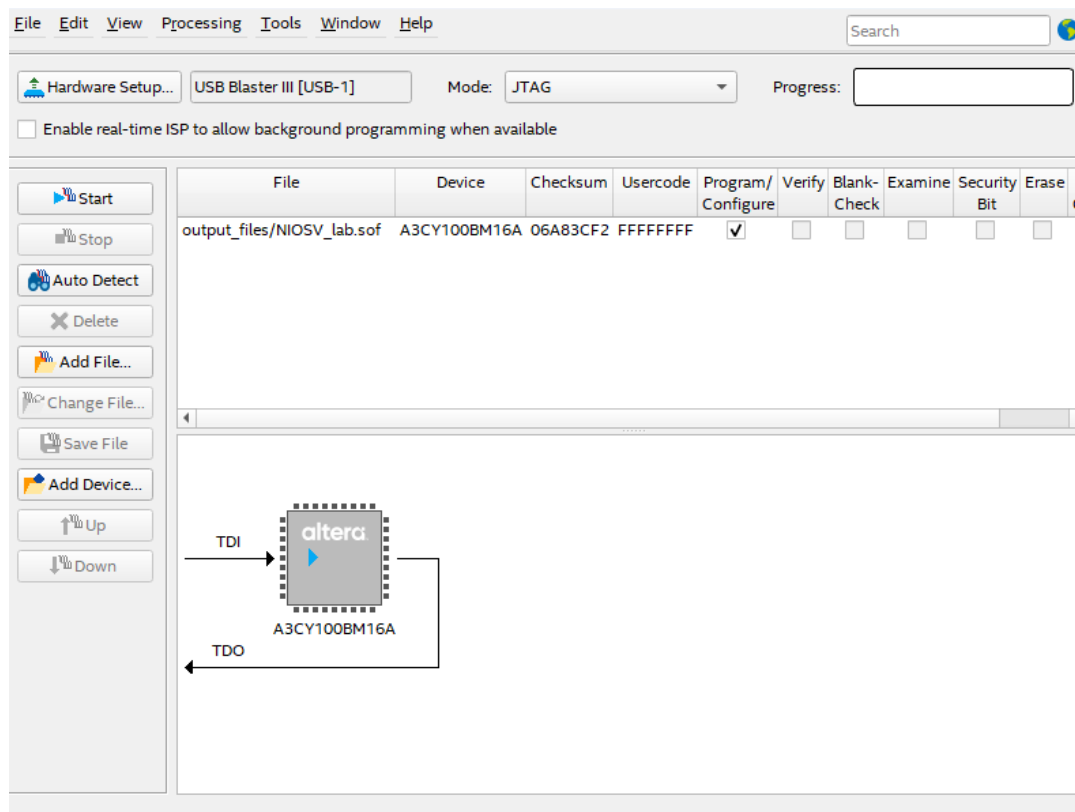


Figure 7-1 : Quartus Programmer

7.2.2 Click the **Hardware Setup** button and double-click on the **USB Blaster III**, then click **Close**.

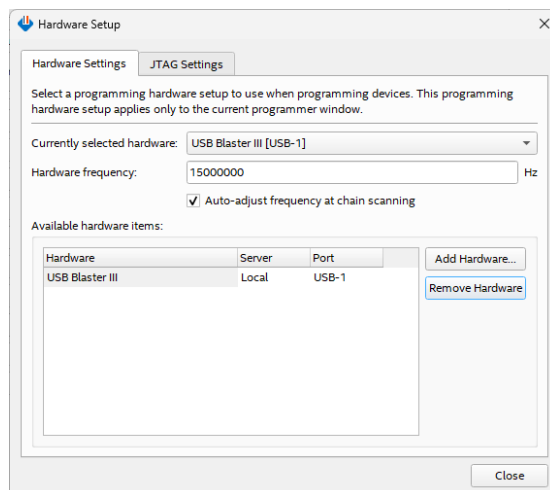


Figure 7-2 : USB Blaster III Hardware Setup

7.2.3 Ensure that the **USB Blaster III** is selected, and the mode is set to **JTAG**.

If the .sof file is not already selected, click on the **Change File** button and navigate to the **output_files/NIOSV_lab.sof**.

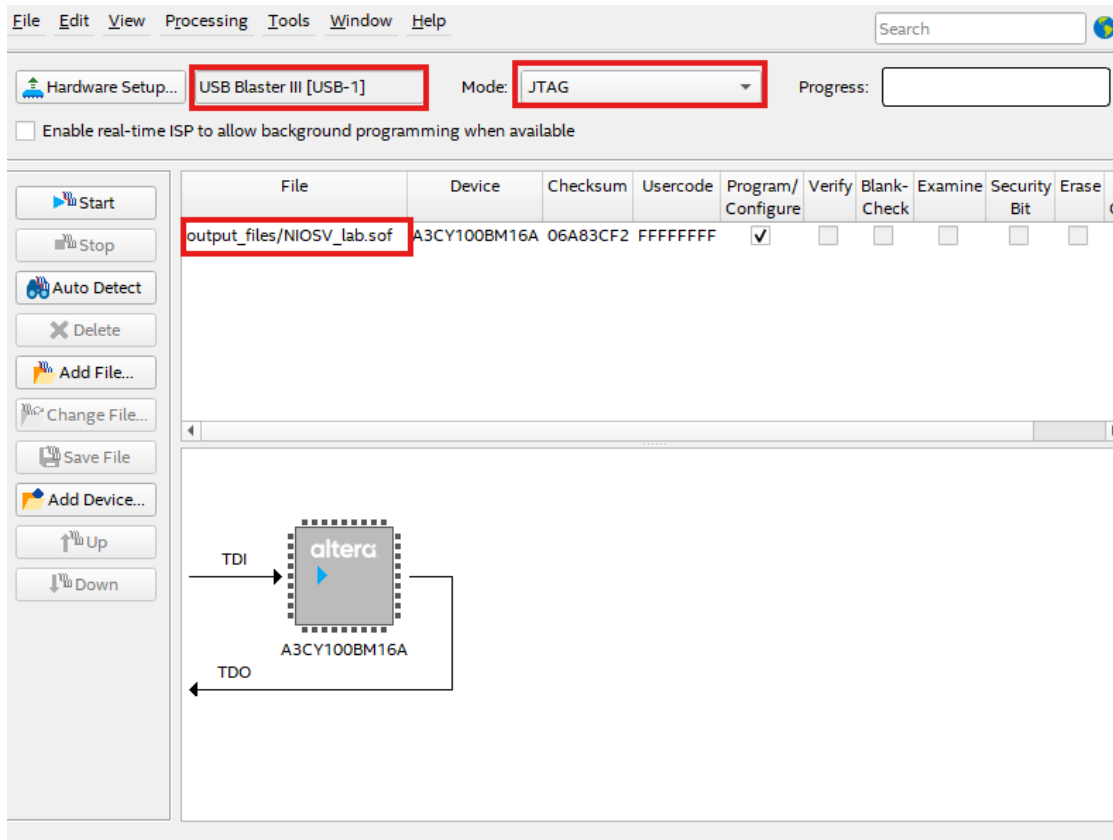


Figure 7-3 : USB Blaster III and .sof Selected

Select the .sof file then click the **Start** button to begin the programming operation and wait for the progress bar to hit 100%.

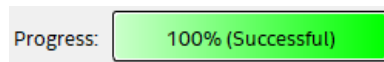


Figure 7-4 : Programmer progress bar

7.3 Open a Terminal Emulator

Open a terminal emulator (Tera term or Putty). Select the COM port shown earlier. Set the emulator to have 8 databits, No stop bits and one Parity bit. Set the baud rate to 115,200.

Observe the “**Hello World**” message printed.

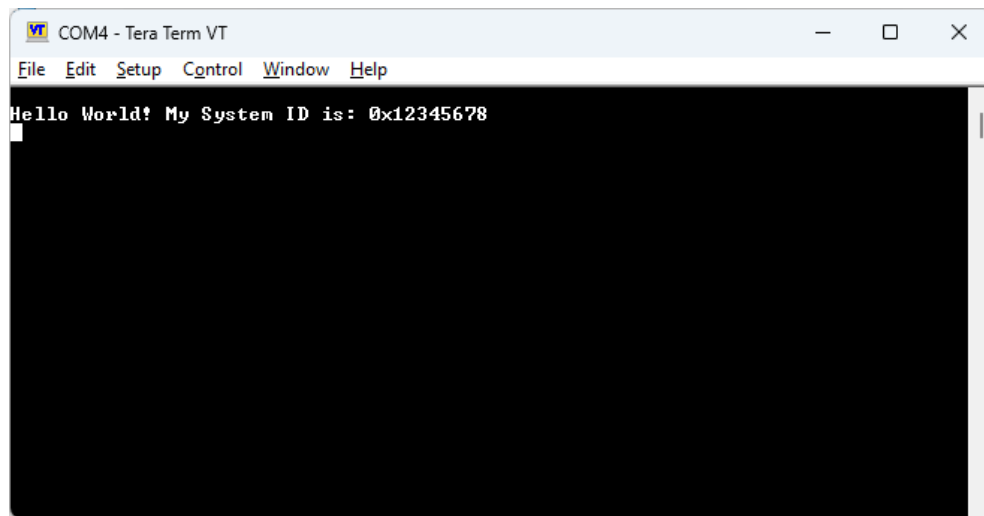
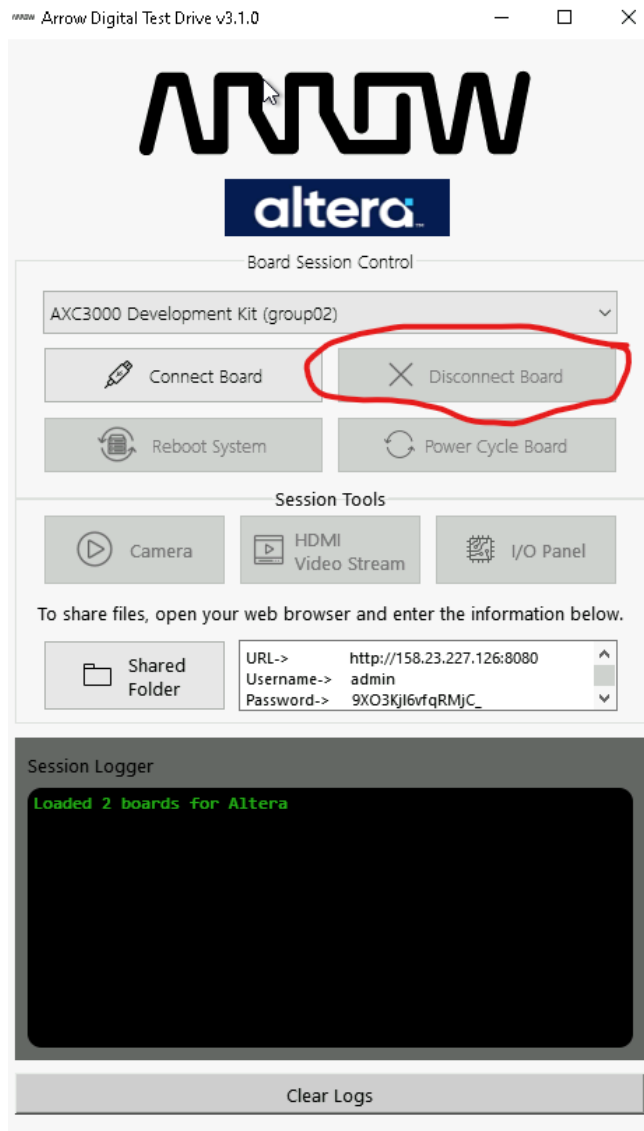


Figure 7-5 : Terminal Emulator

- 7.3.1.1 Please disconnect the board when debugging is complete. This will free up the board resource for others to use.



**CONGRATULATIONS! YOU HAVE SUCCESSFULLY COMPLETED
THE Nios V DESIGN LAB!**

8 Legal Disclaimer

ARROW ELECTRONICS

EVALUATION BOARD LICENSE AGREEMENT

By using this evaluation board or kit (together with all related software, firmware, components, and documentation provided by Arrow, "Evaluation Board"), You ("You") are agreeing to be bound by the terms and conditions of this Evaluation Board License Agreement ("Agreement"). Do not use the Evaluation Board until You have read and agreed to this Agreement. Your use of the Evaluation Board constitutes Your acceptance of this Agreement.

PURPOSE

The purpose of this evaluation board is solely intended for evaluation purposes. Any use of the Board beyond these purposes is on your own risk. Furthermore, according to the applicable law, the offering Arrow entity explicitly does not warrant, guarantee or provide any remedies to you with regard to the board.

LICENSE

Arrow grants You a non-exclusive, limited right to use the enclosed Evaluation Board offering limited features only for Your evaluation and testing purposes in a research and development setting. Usage in a live environment is prohibited. The Evaluation Board shall not be, in any case, directly or indirectly assembled as a part in any production of Yours as it is solely developed to serve evaluation purposes and has no direct function and is not a finished product.

EVALUATION BOARD STATUS

The Evaluation Board offers limited features allowing You only to evaluate and test purposes. The Evaluation Board is not intended for consumer or household use. You are not authorized to use the Evaluation Board in any production system, and it may not be offered for sale or lease, or sold, leased or otherwise distributed for commercial purposes.

OWNERSHIP AND COPYRIGHT

Title to the Evaluation Board remains with Arrow and/or its licensors. This Agreement does not involve any transfer of intellectual property rights ("IPR") for evaluation board. You may not remove any copyright or other proprietary rights notices without prior written authorization from Arrow or its licensors.

RESTRICTIONS AND WARNINGS

Before You handle or use the Evaluation Board, You shall comply with all such warnings and other instructions and employ reasonable safety precautions in using the Evaluation Board. Failure to do so may result in death, personal injury, or property damage.

You shall not use the Evaluation Board in any safety critical or functional safety testing, including but not limited to testing of life supporting, military or nuclear

applications. Arrow expressly disclaims any responsibility for such usage which shall be made at Your sole risk.

WARRANTY

Arrow warrants that it has the right to provide the evaluation board to you. This warranty is provided by Arrow in lieu of all other warranties, written or oral, statutory, express or implied, including any warranty as to merchantability, non-infringement, fitness for any particular purpose, or uninterrupted or error-free operation, all of which are expressly disclaimed. The evaluation board is provided “as is” without any other rights or warranties, directly or indirectly.

You warrant to Arrow that the evaluation board is used only by electronics experts who understand the dangers of handling and using such items, you assume all responsibility and liability for any improper or unsafe handling or use of the evaluation board by you, your employees, affiliates, contractors, and designees.

LIMITATION OF LIABILITIES

In no event shall Arrow be liable to you, whether in contract, tort (including negligence), strict liability, or any other legal theory, for any direct, indirect, special, consequential, incidental, punitive, or exemplary damages with respect to any matters relating to this agreement. In no event shall arrow’s liability arising out of this agreement in the aggregate exceed the amount paid by you under this agreement for the purchase of the evaluation board.

IDENTIFICATION

You shall, at Your expense, defend Arrow and its Affiliates and Licensors against a claim or action brought by a third party for infringement or misappropriation of any patent, copyright, trade secret or other intellectual property right of a third party to the extent resulting from (1) Your combination of the Evaluation Board with any other component, system, software, or firmware, (2) Your modification of the Evaluation Board, or (3) Your use of the Evaluation Board in a manner not permitted under this Agreement. You shall indemnify Arrow and its Affiliates and Licensors against and pay any resulting costs and damages finally awarded against Arrow and its Affiliates and Licensors or agreed to in any settlement, provided that You have sole control of the defense and settlement of the claim or action, and Arrow cooperates in the defense and furnishes all related evidence under its control at Your expense. Arrow will be entitled to participate in the defense of such claim or action and to employ counsel at its own expense.

RECYCLING

The Evaluation Board is not to be disposed as an urban waste. At the end of its life cycle, differentiated waste collection must be followed, as stated in the directive 2002/96/EC. In all the countries belonging to the European Union (EU Dir. 2002/96/EC) and those following differentiated recycling, the Evaluation Board is subject to differentiated recycling at the end of its life cycle, therefore: It is forbidden

to dispose the Evaluation Board as an undifferentiated waste or with other domestic wastes. Consult the local authorities for more information on the proper disposal channels. An incorrect Evaluation Board disposal may cause damage to the environment and is punishable by the law.